



MINISTERIO
DE INDUSTRIA, TURISMO
Y COMERCIO



inteco



Instituto Nacional
de Tecnologías
de la Comunicación

Código Activo: IS-TEOR-VyV01

Nombre Activo: Guía de Validación y Verificación

Ver. Activo: 1.01 [26/10/11]

Copia de Versión Externa: Inteco ver. primera nov/09

GUÍA DE VALIDACIÓN Y VERIFICACIÓN

**Laboratorio Nacional de Calidad del
Software**

El **Instituto Nacional de Tecnologías de la Comunicación, S.A. (INTECO)**, es una sociedad estatal adscrita al Ministerio de Industria, Turismo y Comercio a través de la Secretaría de Estado de Telecomunicaciones y para la Sociedad de la Información.

INTECO tiene la vocación de ser un centro de desarrollo de carácter innovador y de interés público a nivel nacional que constituye una iniciativa enriquecedora y difusora de las nuevas tecnologías en España en clara sintonía con Europa.

Su objetivo fundamental es servir como instrumento para desarrollar la Sociedad de la Información, con actividades propias en el ámbito de la innovación y el desarrollo de proyectos asociados a las Tecnologías de la Información y la Comunicación (TIC), basándose en tres pilares fundamentales: la investigación aplicada, la prestación de servicios y la formación.

La misión de INTECO es aportar valor e innovación a los ciudadanos, a las PYMES, a las Administraciones Públicas y al sector de las tecnologías de la información, a través del desarrollo de proyectos que contribuyan a reforzar la confianza en los servicios de la Sociedad de la Información en nuestro país, promoviendo además una línea de participación internacional.

Para ello, INTECO desarrolla actuaciones en las siguientes líneas:

Seguridad Tecnológica: INTECO está comprometido con la promoción de servicios de la Sociedad de la Información cada vez más seguros, que protejan los datos personales de los interesados, su intimidad, la integridad de su información y eviten ataques que pongan en riesgo los servicios prestados. Y por supuesto que garanticen un cumplimiento estricto de la normativa legal en materia de TIC. Para ello coordina distintas iniciativas públicas en torno a la seguridad de las TIC, que se materializan en la prestación de servicios por parte del Observatorio de la Seguridad de la Información, el Centro Demostrador de Tecnologías de Seguridad, el Centro de Respuesta a Incidentes de Seguridad en Tecnologías de la Información (INTECO-CERT) y la Oficina de Seguridad del Internauta (OSI), de los que se benefician ciudadanos, PYMES, Administraciones Públicas y el sector tecnológico.

Accesibilidad: INTECO promueve servicios de la Sociedad de la Información más accesibles, que supriman las barreras de exclusión, cualquiera que sea la dificultad o carencia técnica, formativa, etc., incluso discapacidad, que tengan sus usuarios. Y que faciliten la integración progresiva de todos los colectivos de usuarios, de modo que todos ellos puedan beneficiarse de las oportunidades que ofrece la Sociedad de la Información. Asimismo desarrolla proyectos en el ámbito de la accesibilidad orientados a garantizar el derecho de ciudadanos y empresas a relacionarse electrónicamente con las AA.PP.

Calidad TIC. INTECO promueve unos servicios de la Sociedad de la Información que cada vez sean de mayor calidad, que garanticen unos adecuados niveles de servicio, lo cual se traduce en una mayor robustez de aplicaciones y sistemas, un compromiso en la disponibilidad y los tiempos de respuesta, un adecuado soporte para los usuarios, una información precisa y clara sobre la evolución de las funcionalidades de los servicios, y en

resumen, servicios cada vez mejores. En esta línea impulsa la competitividad de la industria del Software a través de la promoción de la mejora de la calidad y la certificación de las empresas y profesionales de la ingeniería del software.

Formación: la formación es un factor determinante para la atracción de talento y para la mejora de la competitividad de las empresas. Por ello, INTECO impulsa la formación de universitarios y profesionales en las tecnologías más demandadas por la industria.

NOTA DE EDICIÓN

Esta guía ha sido desarrollada por el Laboratorio Nacional de Calidad del Software de INTECO. Esta primera versión ha sido editada en Noviembre del 2009.

Copyright © 2009 Instituto Nacional de Tecnologías de la comunicación (INTECO)



El presente documento está bajo la licencia Creative Commons Reconocimiento-No comercial-Compartir Igual versión 2.5 España.

Usted es libre de:

- copiar, distribuir y comunicar públicamente la obra
- hacer obras derivadas

Bajo las condiciones siguientes:

- **Reconocimiento.** Debe reconocer los créditos de la obra de la manera especificada por el autor o el licenciador (pero no de una manera que sugiera que tiene su apoyo o apoyan el uso que hace de su obra).
- **No comercial.** No puede utilizar esta obra para fines comerciales.
- **Compartir bajo la misma licencia.** Si altera o transforma esta obra, o genera una obra derivada, sólo puede distribuir la obra generada bajo una licencia idéntica a ésta.

Al reutilizar o distribuir la obra, tiene que dejar bien claro los términos de la licencia de esta obra.

Alguna de estas condiciones puede no aplicarse si se obtiene el permiso del titular de los derechos de autor

Nada en esta licencia menoscaba o restringe los derechos morales del autor.

Esto es un resumen legible por humanos del texto legal (la licencia completa) disponible en <http://creativecommons.org/licenses/by-nc-sa/2.5/es/>

El presente documento cumple con las condiciones de accesibilidad del formato PDF (Portable Document Format).

Se trata de un documento estructurado y etiquetado, provisto de alternativas a todo elemento no textual, marcado de idioma y orden de lectura adecuado.

Para ampliar información sobre la construcción de documentos PDF accesibles puede consultar la guía disponible en la sección [Accesibilidad > Formación > Manuales y Guías](#) de la página <http://www.inteco.es>.

AVISO LEGAL

- CMMI® es una marca registrada en la Oficina de Marcas y Patentes de EEUU por la Universidad Carnegie Mellon
- Las distintas normas ISO mencionadas han sido desarrolladas por la International Organization for Standardization.

Todas las demás marcas registradas que se mencionan, usan o citan en la presente guía son propiedad de los respectivos titulares.

INTECO cita estas marcas porque se consideran referentes en los temas que se tratan, buscando únicamente fines puramente divulgativos. En ningún momento INTECO busca con su mención el uso interesado de estas marcas ni manifestar cualquier participación y/o autoría de las mismas.

Nada de lo contenido en este documento debe ser entendido como concesión, por implicación o de otra forma, y cualquier licencia o derecho para las Marcas Registradas deben tener una autorización escrita de los terceros propietarios de la marca.

Por otro lado, INTECO renuncia expresamente a asumir cualquier responsabilidad relacionada con la publicación de las Marcas Registradas en este documento en cuanto al uso de ninguna en particular y se eximen de la responsabilidad de la utilización de dichas Marcas por terceros.

El carácter de todas las guías editadas por INTECO es únicamente formativo, buscando en todo momento facilitar a los lectores la comprensión, adaptación y divulgación de las disciplinas, metodologías, estándares y normas presentes en el ámbito de la calidad del software.

ÍNDICE

1.	INTRODUCCIÓN	10
2.	VALIDACIÓN Y VERIFICACIÓN EN CMMI®	11
2.1.	Metas y prácticas específicas	12
2.1.1.	Validación	12
2.1.2.	Verificación	13
2.2.	Validación y Verificación de la adquisición	14
2.2.1.	Validación	14
2.2.2.	Verificación	15
2.3.	Áreas de proceso relacionadas	15
3.	PROCESOS DE VALIDACIÓN Y VERIFICACIÓN	16
3.1.	Validación y verificación en el ciclo de vida	17
3.1.1.	Modelo en V	17
3.1.2.	Modelo incremental y evolutivo	20
3.1.3.	Modelo en espiral	21
3.1.4.	Modelo de Prototipos	22
3.2.	Actividades de validación y verificación	23
3.2.1.	Pruebas	24
3.2.2.	Estrategia de pruebas	30
3.2.3.	Niveles de pruebas	36
3.2.4.	Buenas prácticas en las pruebas del software	48
3.2.5.	Tipos de pruebas	49
3.2.6.	Revisiones	52
4.	TÉCNICAS Y HERRAMIENTAS DE VALIDACIÓN Y VERIFICACIÓN	56
4.1.	Técnicas de pruebas	56
4.1.1.	Técnicas de pruebas dinámicas	56
4.1.2.	Técnicas de control estáticas	61
4.2.	Herramientas de pruebas	67
4.2.1.	Beneficios del uso de herramientas	67
4.2.2.	Riesgos del uso de herramientas	68
4.2.3.	Clasificación de herramientas de pruebas	69
5.	ESTÁNDARES DE REFERENCIA	74
5.1.	IEEE Std. 1012-2004	74

5.2.	IEEE Std. 829-2008	76
5.3.	ISO/IEC 29119	77
6.	GLOSARIO	78
7.	ACRÓNIMOS	80
8.	REFERENCIAS	81

ÍNDICE FIGURAS

Figura 1. Constelaciones CMMI®	11
Figura 2. Actividades de validación y verificación en el modelo en V.....	18
Figura 3. Modelo incremental	21
Figura 4. Modelo espiral	22
Figura 5. Modelo de ciclo de vida de prototipo	23
Figura 6. Proceso de pruebas en el ciclo de vida de un producto.....	25
Figura 7. Flujo del Proceso de pruebas	27
Figura 8. Relación componentes sistema de pruebas	29
Figura 9. La automatización como parte de la estrategia de pruebas	35
Figura 10. Flujo de control pruebas unitarias.....	39
Figura 11. Flujo de control pruebas de integración	41
Figura 12. Flujo de control pruebas de sistema	45
Figura 13. Flujo de control pruebas de aceptación	47
Figura 14. Revisiones durante el desarrollo de un producto	52
Figura 15. Clasificación técnicas dinámicas	57

ÍNDICE TABLAS

Tabla 1. E/S y roles en las pruebas unitarias.....	38
Tabla 2. E/S, tareas y roles en las pruebas de integración	40
Tabla 3. E/S, tareas y roles pruebas de sistema.....	43
Tabla 4. E/S, tareas y roles de pruebas de aceptación.....	46
Tabla 5. Entregables afectados por las técnicas estáticas.....	62

1. INTRODUCCIÓN

La guía de Validación y Verificación está orientada a las áreas de proceso de CMMI®, tanto de la constelación de desarrollo (Validación y Verificación) como de la de adquisición (Validación y Verificación de la adquisición). Pretende ser una guía de apoyo a la implementación de dichas áreas de proceso, y de sus metas y actividades.

En la primera parte de la guía se situarán estas áreas de proceso dentro del modelo de CMMI® y se verá la relación que tienen con otras áreas de proceso del modelo.

A continuación se describirán en detalle los procesos de validación y verificación. Primero se situarán dichos procesos sobre ciertos modelos de ciclo de vida para, a continuación, describir las diferentes actividades relacionadas con dichos procesos. Se hará un estudio de las pruebas, definiendo posibles estrategias, niveles y tipos de pruebas, buenas prácticas, y por otra parte de las revisiones.

Tras la definición de las actividades de validación y verificación se propondrán una serie de técnicas a seguir y herramientas a utilizar para facilitar el desarrollo de las tareas relacionadas con dichos procesos.

Por último se hará referencia a ciertos estándares relacionados con dichas áreas de proceso útiles para tener otra perspectiva de los procesos.

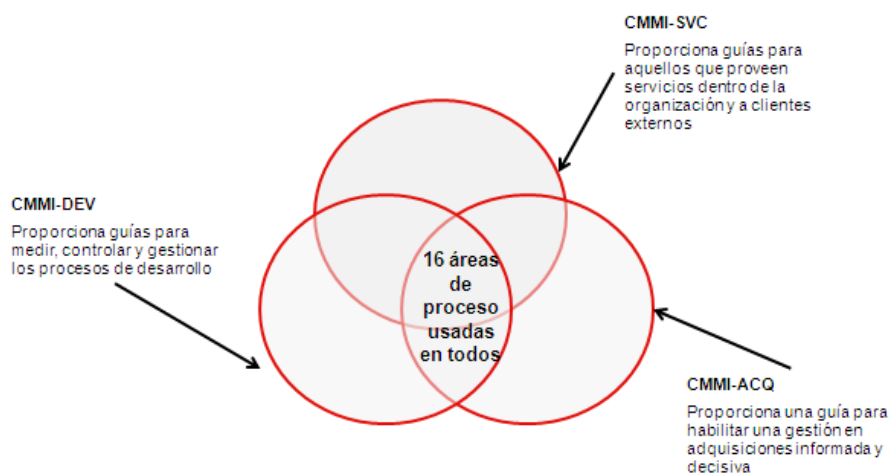
2. VALIDACIÓN Y VERIFICACIÓN EN CMMI®

Capability Maturity Model Integration (CMMI®) es un modelo para la mejora de procesos que proporciona a las organizaciones los elementos esenciales para desarrollar procesos eficaces.

Los componentes de CMMI® están organizados en agrupaciones llamadas constelaciones, cada una de ellas orientada a un área de interés:

- CMMI® for Development (CMMI-DEV®).
- CMMI® for Services (CMMI-SVC®).
- CMMI® for Acquisition (CMMI-ACQ®).

Figura 1. Constelaciones CMMI®



Tanto la constelación de desarrollo como la de adquisición contemplan las áreas de proceso Validación y Verificación.

En la constelación de desarrollo (CMMI-DEV®), el modelo CMMI® incluye las áreas de proceso Validación y Verificación en la categoría de **Ingeniería**. Estas áreas de proceso son necesarias, pero no suficientes, para conseguir el nivel de madurez 3, o nivel definido, al igual que ocurre en la constelación CMMI-ACQ®.

A diferencia de CMMI-DEV®, en la constelación CMMI-ACQ® las áreas de proceso Validación y Verificación pertenecen a la categoría de **Adquisición**.

En ambas constelaciones, tanto las metas como las prácticas que propone el modelo CMMI® para dichas áreas de proceso son similares, aunque para la constelación de adquisiciones hay algunos matices significativos propios de dicha constelación que se describen en el apartado 2.2 de esta guía: "Validación y Verificación de la adquisición".

2.1. METAS Y PRÁCTICAS ESPECÍFICAS

A continuación, se describen las metas y prácticas específicas que propone el modelo CMMI® para cubrir las áreas de proceso de Validación y Verificación, tanto en la constelación CMMI-DEV® como en la constelación CMMI-ACQ®.

A lo largo de la guía se expondrán métodos, recomendaciones y buenas prácticas para poder implementar las distintas metas y prácticas que se mencionan a continuación.

2.1.1. Validación

El propósito de la Validación en CMMI® es demostrar que un producto o componente del mismo satisface el uso para el que se creó al situarlo sobre el entorno previsto. Las dos metas que se pretenden cumplir en esta área son:

SG 1 Preparar la validación

Las actividades de preparación de la validación incluyen la selección de productos o componentes a validar, establecer y mantener el entorno de validación, procedimientos y criterios. Cualquier producto o componente del producto puede ser objeto de validación, incluyendo mantenimiento, sustituciones o productos de formación.

Los entornos usados para realizar la integración y verificación de los productos pueden compartirse con las actividades de validación y así reducir costes y mejorar la eficiencia y la productividad.

Las prácticas relacionadas con esta meta son:

SP 1.1 Seleccionar productos a validar: Seleccionar los productos y componentes de productos a validar y los métodos de validación que van a aplicarse.

SP 1.2 Establecer un entorno de validación: Establecer y mantener el entorno necesario que soporte la validación.

SP 1.3 Establecer procedimientos y criterios de validación: Establecer y mantener procedimientos y criterios de validación que estén alineados con las características de los productos seleccionados, las restricciones del cliente sobre validación, los métodos y el entorno de validación.

SG 2 Validar los productos y los componentes de los productos

Los productos y componentes de productos son validados para asegurar que son apropiados para usarse en su entorno operativo.

Las actividades de validación se realizan a través del ciclo de vida del producto, y las prácticas a realizar son:

SP 2.1 Realizar validación: Esta práctica permite la realización de la validación de acuerdo a los métodos, los procedimientos y los criterios.

SP 2.2 Analizar los resultados de las actividades de validación

2.1.2. Verificación

El propósito de la Verificación en CMMI® es asegurar que los productos seleccionados cumplen los requisitos especificados. Esta área de proceso tiene tres metas:

SG 1 Preparación de la verificación

Para preparar la verificación las prácticas a seguir son:

SP 1.1 Seleccionar productos para la verificación: Permite la identificación de los productos de trabajo a verificar, los métodos a usar para realizar la verificación y los requisitos a satisfacer por cada producto de trabajo seleccionado.

SP 1.2 Establecer el entorno de verificación: Permite establecer y mantener el entorno que será usado para llevar a cabo la verificación.

SP 1.3 Establecer criterios y procedimientos de verificación: Permite el desarrollo de los procedimientos y de los criterios de verificación que están alineados con los productos de trabajo, requisitos, métodos y características seleccionados del entorno de verificación.

SG 2 Realización de revisiones entre pares

Las revisiones entre pares incluyen un examen metodológico de los productos para identificar posibles defectos del producto o recomendar cambios necesarios. Las revisiones entre pares se aplican sobre los productos seleccionados, y las prácticas a seguir son:

SP 2.1 Preparar revisiones entre pares.

SP 2.2 Dirigir revisiones entre pares: conducir las revisiones entre pares e identificar temas resultantes de la revisión.

SP 2.3 Analizar los datos de las revisiones entre pares: analizar los datos obtenidos de las prácticas anteriores.

SG 3 Verificar la selección de productos

La verificación de los métodos, procedimientos y criterios se usa para verificar los productos seleccionados y cualquier mantenimiento asociado, formación y servicios de soporte, usando para ello el entorno de verificación apropiado.

SP 3.1 Realizar la verificación de los productos seleccionados.

SP 3.2 Analizar los resultados de las actividades de verificación.

2.2. VALIDACIÓN Y VERIFICACIÓN DE LA ADQUISICIÓN

En este apartado se van a describir los matices que la constelación CMMI-ACQ® da a las áreas de proceso Validación y Verificación, y que las diferencian de la constelación de desarrollo.

2.2.1. Validación

Según CMMI-ACQ®, la validación demuestra que el producto o servicio adquirido satisface las necesidades de las partes interesadas en el negocio y los requisitos del cliente.

Las actividades de validación son llevadas a cabo por el adquiridor, el proveedor o por ambas partes en función de lo establecido en el acuerdo con el proveedor.

En el acuerdo con el proveedor normalmente se recogen los distintos aspectos de la validación:

- Lista de productos adquiridos a validar por el adquiridor antes de la aceptación formal.
- Lista de productos a validar por el proveedor junto con los clientes, usuarios y demás involucrados en el negocio, utilizando estándares, procedimientos, métodos y herramientas de validación aplicables.
- Métricas a recoger y a proporcionar por el proveedor con respecto a las actividades de validación.
- Rol que tendrá el proveedor en la validación del producto o componentes del producto.
- Entornos de validación a utilizar por el adquiridor.
- Procedimientos y criterios de validación aplicables al proveedor.

Los productos, o componentes de productos, son seleccionados para ser validados por el adquiridor dependiendo de los atributos del proyecto.

La selección de los métodos más apropiados para llevar a cabo la validación se basa en elegir el mejor procedimiento para predecir cómo de bien el producto o servicio adquirido satisfará las necesidades de las partes involucradas en el negocio.

2.2.2. Verificación

La verificación de la adquisición establece si los productos adquiridos reflejan los requisitos especificados. Al igual que en la constelación de desarrollo, la verificación es un proceso incremental que ocurre durante todo el proceso de adquisición del producto o servicio.

En CMMI-ACQ®, las actividades de verificación normalmente incluyen:

- Revisión del paquete de solicitud.
- Planes y acuerdos con proveedores.
- Documentos de requisitos.
- Restricciones de diseño desarrolladas por el adquiridor.
- Otros productos de trabajo desarrollados por el adquiridor.

Los proveedores deberían estar involucrados en la selección de los métodos de verificación para asegurarse de que son apropiados para el entorno de los proveedores.

2.3. ÁREAS DE PROCESO RELACIONADAS

Las áreas de proceso de validación y verificación están relacionadas con otras áreas de proceso del modelo CMMI®.

Por ejemplo, para validar los requisitos, aparte del área de “Validación” se podría consultar el área de “Desarrollo de requisitos”. Y para obtener más información sobre la gestión de requisitos se tomaría como referencia el área de proceso “Gestión de requisitos”.

Para más información sobre la transformación de los requisitos en especificaciones de producto, y para tomar acciones correctivas cuando se identifican problemas de validación que afectan al diseño del producto o del componente del producto, habría que acudir al área de proceso de “Solución Técnica”.

3. PROCESOS DE VALIDACIÓN Y VERIFICACIÓN

El proceso de desarrollo adoptado por un proyecto dependerá de los objetivos y metas que tenga el proyecto. Existen distintos modelos de ciclo de vida que se han desarrollado para conseguir diferentes objetivos, pero en cualquiera de ellos será necesario un proceso que asegure la calidad a través de todo el ciclo de vida de desarrollo del producto. Al final de cada fase del ciclo de vida debería comprobarse que el trabajo realizado hasta ese momento cumple con todos los objetivos previstos. Este es el punto clave, en el que tiene lugar la evaluación del producto, donde se decide si está o no preparado para pasar a la siguiente fase. De esta forma, si hay errores y son detectados, será más eficiente corregirlos que si se descubriesen en etapas más avanzadas. Este tipo de comprobaciones a lo largo del ciclo de vida incluyen dos actividades: la **verificación** y la **validación**.

La validación y la verificación son procesos de evaluación de productos que son útiles para determinar si se satisfacen las necesidades del negocio y si se están construyendo acorde a las especificaciones.

La verificación y la validación no son lo mismo, aunque a menudo se confunden. [\[3\]](#) Boehm expresó la diferencia entre ellas de la siguiente manera:

- Verificación: “¿Se está construyendo el producto de la manera correcta?”
- Validación: “¿Se está construyendo el producto correcto?”

El papel de la verificación implica comprobar que el software está de acuerdo con su especificación. Debería comprobarse que satisface sus requisitos funcionales y no funcionales.

La validación, sin embargo, tiene como objetivo asegurar que el sistema satisface las expectativas del cliente. Va más allá de la comprobación de que el sistema satisface su especificación para demostrar que el software hace lo que el cliente espera que haga, ya que las especificaciones del sistema no siempre reflejan los deseos o necesidades reales de los usuarios y los propietarios del sistema.

El objetivo último del proceso de verificación y validación es comprobar que el sistema está hecho para un propósito. Esto significa que el sistema debe ser lo suficientemente bueno para su uso previsto. El nivel de confianza requerido depende de:

- **El propósito o función del sistema.** El nivel de confianza necesario depende de lo crítico que sea el software para una organización. Por ejemplo, el nivel de confianza del software que se utiliza para controlar un sistema de seguridad crítico es mucho más alto que el requerido para un prototipo de un sistema software que ha sido desarrollado para demostrar nuevas ideas.
- **Las expectativas del usuario.** Una reflexión lamentable sobre la industria del software es que a muchos usuarios no les sorprende cuando su software falla

durante su uso. Están dispuestos a aceptar estos fallos del sistema cuando los beneficios de su uso son mayores que sus desventajas. Sin embargo, la tolerancia de los usuarios a los fallos de los sistemas está decreciendo en los últimos años. Actualmente es menos aceptable entregar sistemas no fiables, por lo que las compañías deben invertir más esfuerzo en llevar a cabo las actividades de verificación y validación.

- **Entorno de mercado actual.** Cuando un sistema se comercializa, los vendedores del sistema deben tener en cuenta los programas competidores, el precio que sus clientes están dispuestos a pagar por el sistema y los plazos requeridos para entregar dicho sistema. Cuando una compañía tiene pocos competidores, puede decidir entregar un programa antes de que haya sido completamente probado y depurado, debido a que quiere ser el primero en el mercado. Cuando los clientes no están dispuestos a pagar precios altos por el software, pueden estar dispuestos a tolerar más defectos en él. Todos estos factores pueden considerarse cuando se decide cuánto esfuerzo debería invertirse en el proceso de validación y verificación.

3.1. VALIDACIÓN Y VERIFICACIÓN EN EL CICLO DE VIDA

La validación y la verificación son procesos costosos y para algunos sistemas, tales como los sistemas de tiempo real con complejas restricciones no funcionales, más de la mitad del presupuesto para el desarrollo del sistema puede invertirse en ambos procesos. Es necesario que haya una planificación cuidadosa para obtener el máximo provecho de las inspecciones y pruebas, y controlar los costes de los procesos de validación y verificación. Dicha planificación debería comenzar en etapas tempranas del proceso de desarrollo, como se verá, con la ayuda de los modelos de ciclo de vida.

Existen distintos modelos que representan el ciclo de vida del producto. El **modelo en cascada** es uno de los más sencillos en cuanto a su diseño. Es el enfoque metodológico que ordena rigurosamente las etapas del ciclo de vida del software, de forma que el inicio de cada etapa debe esperar a la finalización de la inmediatamente anterior.

En este modelo, las pruebas se realizan al final del ciclo de vida del proyecto por lo que los defectos son detectados cerca de la fecha de implementación del producto o sistema, lo que supone un coste muy elevado en la corrección de defectos. El modelo en cascada es un modelo tradicional, pero existen otros modelos que se ajustan mejor a los proyectos actuales y que se explican a continuación.

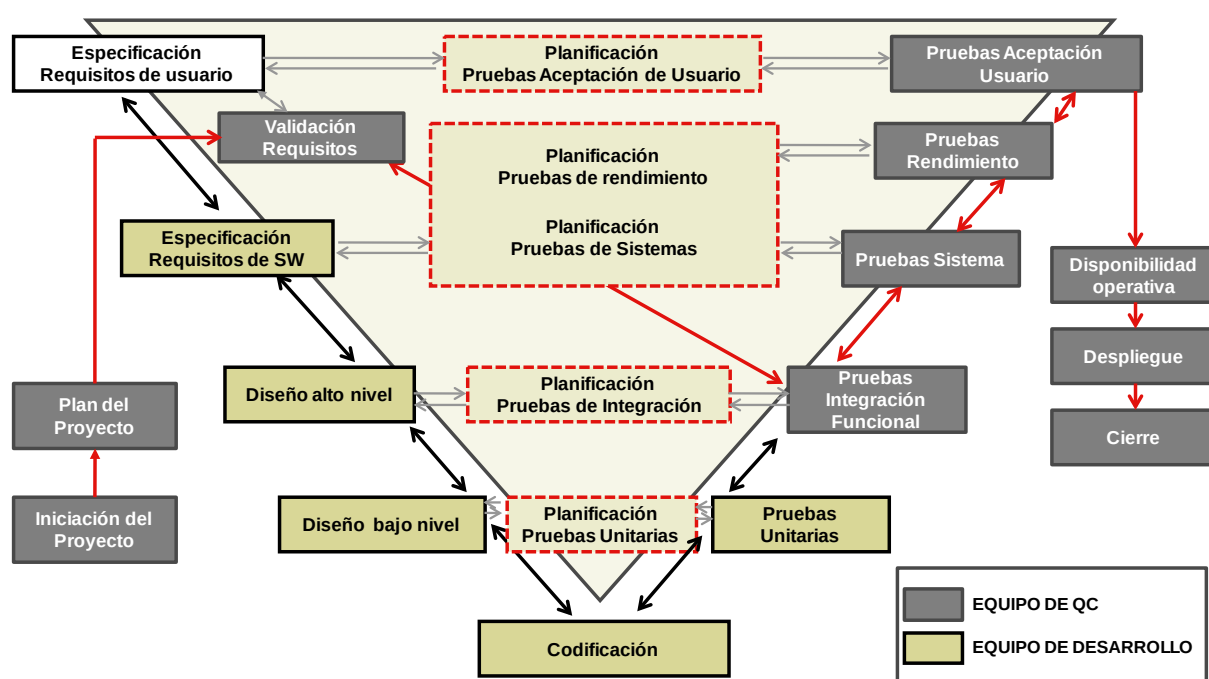
3.1.1. Modelo en V

El **modelo en V** fue desarrollado para solventar algunos problemas que ocurrían usando el enfoque propuesto por el modelo tradicional de cascada. Los defectos se encontraban demasiado tarde en el ciclo de vida, ya que las pruebas no se introducían hasta el final del proyecto. El modelo en V proporciona guías para comenzar con la realización de pruebas tan pronto como sea posible en el ciclo de vida del producto. Hay una variedad de actividades que se han de realizar antes del fin de la fase de codificación. Estas actividades

deberían ser llevadas a cabo en paralelo con las actividades de desarrollo, y los técnicos de pruebas necesitan trabajar con los desarrolladores y analistas de negocio de tal forma que puedan realizar estas actividades y tareas y producir una serie de entregables de pruebas. Los productos de trabajo generados por los desarrolladores y analistas de negocio durante el desarrollo son las bases de las pruebas en uno o más niveles.

El modelo en V es un modelo que ilustra cómo las actividades de prueba (verificación y validación) se pueden integrar en cada fase del ciclo de vida como se puede ver en la siguiente figura.

Figura 2. Actividades de validación y verificación en el modelo en V



Dentro del modelo en V, las pruebas de validación tienen lugar especialmente durante las etapas tempranas, por ejemplo, revisando los requisitos de usuario y durante etapas tardías, por ejemplo, durante las pruebas de aceptación de usuario. A continuación se describe en más detalle cada una de ellas.

La planificación de la verificación y validación del sistema comienza en etapas tempranas del desarrollo. El gráfico muestra cómo durante las fases de iniciación y planificación del proyecto, aparte de las propias tareas relacionadas con estas actividades, se deberían definir los puntos de control, identificar los productos sobre los cuales llevar a cabo el control de calidad y la estrategia de esfuerzo relacionada con estas actividades (control de calidad).

Una vez finalizadas estas actividades se pasaría a la **validación de requisitos**. Esta actividad trata de comprobar que los requisitos realmente definen el sistema que el cliente desea, es decir, que cumple las necesidades del usuario. Los usuarios deben imaginarse el sistema en funcionamiento y cómo encajaría dicho sistema en su trabajo. Para los usuarios

del sistema ésta es una tarea difícil y, como consecuencia, rara vez se encuentran todos los problemas en los requisitos durante el proceso de validación de éstos.

La validación de requisitos es una tarea importante debido a que los errores en el documento de requisitos pueden conducir a importantes costes al repetir el trabajo cuando son descubiertos durante el desarrollo o después de que el sistema esté en uso. También es cierto que, en ocasiones, es inevitable que haya cambios adicionales de requisitos para corregir las omisiones y las malas interpretaciones después de que el documento de requisitos haya sido aprobado.

Durante el proceso de validación de requisitos, se deben llevar a cabo verificaciones sobre el documento de especificación de requisitos. Estas verificaciones comprenden:

- **Verificaciones de validez.** Un usuario puede pensar que se necesita un sistema para llevar a cabo ciertas funciones y, sin embargo, el razonamiento y el análisis pueden identificar que se requieren funciones adicionales o diferentes.
- **Verificaciones de consistencia.** Los requisitos en el documento no deben contradecirse, es decir, no debe haber descripciones contradictorias de la misma función del sistema.
- **Verificaciones de completitud.** La especificación de requisitos debe incluir requisitos que definan todas las funciones y restricciones propuestas por el usuario del sistema.
- **Verificación de realismo.** Utilizando la tecnología existente, los requisitos deben verificarse para asegurar que se pueden implementar. Estas verificaciones también deben tener en cuenta el presupuesto y la planificación del desarrollador del sistema.
- **Verificabilidad.** Para reducir la posibilidad de discusiones entre el cliente y el contratista, los requisitos del sistema siempre deben redactarse de tal forma que sean verificables, es decir, que debe poder crearse un conjunto de pruebas que demuestren que el sistema a entregar cumple cada uno de los requisitos especificados.

Existen diferentes técnicas de validación de requisitos tales como revisiones de requisitos, construcción de prototipos o generación de casos de prueba. Dichas técnicas de validación se verán en apartados posteriores.

Una vez validados los requisitos se debe pasar a la propia **ejecución de pruebas**, desde las pruebas de integración donde se valida el producto frente al diseño, las pruebas de sistema y rendimiento, donde se valida el producto frente a la especificación de requisitos software, hasta las pruebas de aceptación de usuario, que en algunos casos las realiza el propio usuario o el equipo de control de calidad, con el objetivo de validar los requisitos de usuario. Por último, la validación intervendría en actividades de disponibilidad operativa y despliegue del producto en producción para asegurar que tiene el nivel adecuado de calidad. Por otro lado, las pruebas unitarias se deben realizar a medida que se vaya generando código.

Las actividades que aparecen en la parte central del gráfico son la planificación y diseño de las pruebas correspondientes al nivel en el que se está trabajando. Por ejemplo, en la especificación de requisitos se debe planificar qué tipo de pruebas se van a realizar para validar el cumplimiento de estos requisitos, o en el diseño se debe planificar qué pruebas se van a realizar para validar el cumplimiento del diseño.

La verificación encajaría en todas y cada una de las fases del ciclo de vida porque en dichas fases es necesario comprobar que las tareas se están desarrollando de la manera adecuada tal y como se planificaron. Por ejemplo, se deberán realizar auditorías en distintos puntos del ciclo de vida para asegurar que se ejecutan los procesos y se generan los entregables correctos.

3.1.2. Modelo incremental y evolutivo

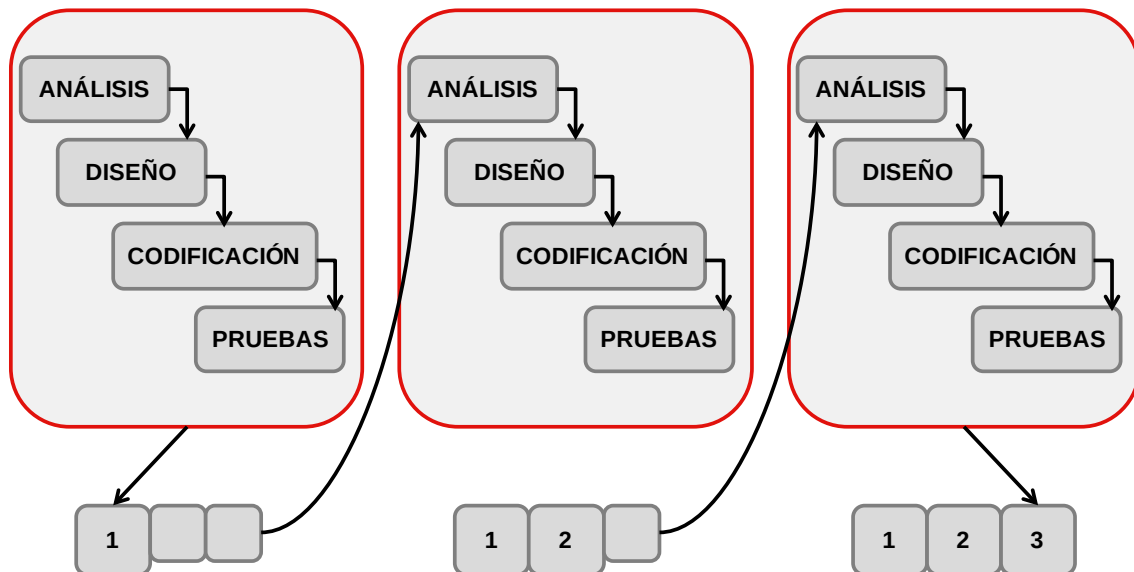
El modelo incremental combina elementos del modelo en cascada con la filosofía interactiva de construcción de prototipos. Se basa en la filosofía de construir el producto incrementando las funcionalidades del programa. Este modelo aplica secuencias lineales de forma escalonada mientras progresa el tiempo en el calendario. Cada secuencia lineal produce un incremento del software.

Este modelo se centra en la entrega de un producto operativo con cada incremento. Cuando se utiliza un modelo incremental, el primer incremento es a menudo un producto esencial, que posee sólo los requisitos básicos. Los primeros incrementos son versiones incompletas del producto final, pero proporcionan al usuario la funcionalidad que precisa y también una plataforma para la evaluación.

En muchos proyectos se adoptan modelos incrementales y evolutivos como el que se muestra en la siguiente figura. En estos casos, el sistema es analizado, diseñado, desarrollado y probado en incrementos claramente definidos. En cualquier punto, después de que el primer incremento se haya realizado, el equipo de proyecto puede entregar por lo menos una porción de la funcionalidad planificada.

Una ventaja del desarrollo incremental es, que una versión probable del sistema, está disponible en etapas tempranas del proceso de desarrollo. Las funcionalidades pueden probarse a medida que se van añadiendo al sistema, por lo que no tiene que realizarse una implementación completa del producto para que comiencen las pruebas.

Figura 3. Modelo incremental



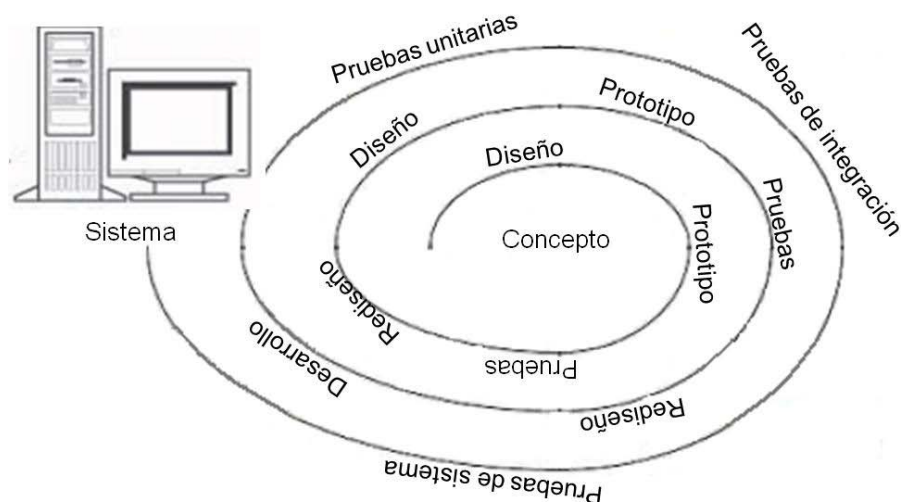
Los enfoques del modelo varían en términos de formalidad desde la “Programación Extrema” al “Desarrollo Rápido de Aplicaciones”. Estos modelos no resolverán todos los problemas de las pruebas, aunque el papel de las pruebas en estos modelos cada vez está evolucionando más.

3.1.3. Modelo en espiral

Los términos modelo evolutivo y modelo incremental se suelen usar indistintamente, aunque se podría destacar un pequeño matiz que los diferencia. Mientras que en el modelo incremental hay una serie de características definidas de arriba a abajo, en el modelo evolutivo las características van evolucionando a lo largo del tiempo.

El modelo en espiral es útil cuando el equipo de proyecto no tiene otra manera de especificar de manera precisa y por adelantado las características del sistema que se va a construir. En este caso se pueden identificar dichas características con la ayuda de prototipos. Los desarrolladores construyen un prototipo inicial y lo prueban. Las pruebas, el re-diseño, y el prototipado deben ser actividades continuas hasta que el desarrollo del conjunto de características haya finalizado.

Figura 4. Modelo espiral



En el modelo en espiral, el primer prototipo puede implicar la realización de pruebas muy pronto en el proyecto, pero esto no implica que el final esté cercano. Tal y como se muestra en el gráfico, por lo general se realizan varios prototipos antes de la entrega del producto final, y por lo tanto se deberá ir modificando el diseño del prototipo inicial hasta que se consiga el producto deseado por el usuario. Por último, se realizarán las pruebas oportunas unitarias, de integración y de sistema.

Haciendo uso de este modelo hay que tener cuidado para no entregar demasiado pronto al usuario un prototipo como producto final que aun no esté listo. En ocasiones, la presión del tiempo hace que ocurra este hecho, en cuyo caso se debería tener en cuenta el análisis de riesgos realizado con anterioridad.

3.1.4. Modelo de Prototipos

El cliente a menudo define un conjunto de objetivos generales para el software, pero no identifica los requisitos detallados de entrada, proceso o salida. En otros casos, el responsable del desarrollo del software puede no estar seguro de la eficiencia de un algoritmo, de la calidad de adaptación de un sistema operativo, o de la forma en que debería realizarse la interacción hombre-máquina. En estas y en otras muchas situaciones, un paradigma de construcción de prototipos puede ofrecer el mejor enfoque.

El paradigma de construcción de prototipos comienza con la recolección de requisitos. El desarrollador y el cliente encuentran y definen los objetivos globales para el software, identifican los requisitos conocidos y las áreas del esquema donde es obligatoria más definición. Entonces entraría el diseño rápido. El diseño rápido se centra en una representación de esos aspectos del software que serán visibles para el usuario/cliente. El diseño rápido lleva a la construcción de un prototipo. El prototipo lo evalúa el cliente/usuario y se utiliza para refinar los requisitos del software a desarrollar. La iteración ocurre cuando el

prototipo se pone a punto para satisfacer las necesidades del cliente, permitiendo al mismo tiempo que el desarrollador comprenda mejor lo que se necesita hacer.

En este modelo está muy presente tanto la validación como la verificación, ya que el desarrollador tiene muy en cuenta la opinión del cliente con el objetivo de conseguir cubrir sus necesidades, realizando revisiones de manera periódica para conseguir un producto final que cumpla los requisitos especificados.

Figura 5. Modelo de ciclo de vida de prototipo



3.2. ACTIVIDADES DE VALIDACIÓN Y VERIFICACIÓN

En este apartado se describirán las principales tareas relacionadas con los procesos de validación y verificación utilizadas para la eliminación de errores, como son las pruebas y las revisiones.

Primeramente se comenzará definiendo el concepto de pruebas, situándolas dentro del ciclo de vida de desarrollo de un producto y relacionándolas con el proceso y sistema de pruebas, conceptos clave en el ámbito de las pruebas.

También se definirán una serie de estrategias a seguir durante el proceso de pruebas, proporcionando pautas que pueden ser útiles para la elección de la mejor estrategia en función de la situación de la empresa. A continuación se identificarán y describirán distintos tipos y niveles de pruebas, para finalizar con una serie de buenas prácticas a seguir durante el proceso de pruebas.

Una vez abarcado el tema de las pruebas, más relacionadas con el proceso de validación, se pasará a la descripción de las revisiones, tarea más orientada al proceso de verificación. En esta parte se resaltarán las revisiones entre pares, ya que son un concepto fundamental en el área de proceso de Verificación del modelo CMMI®. Las revisiones son técnicas de control estáticas y se describirán en más detalle en el apartado relativo a técnicas.

3.2.1. Pruebas

Como se ha visto hasta ahora, las pruebas son una parte imprescindible de los procesos de validación y verificación. Son actividades clave para que dichos procesos tengan éxito, ya que ayudan a entregar el producto con la calidad suficiente para satisfacer las necesidades del cliente y con la certeza de que el producto cumple las especificaciones definidas. Este objetivo conduce a las **pruebas de validación**, en las que se espera que el sistema funcione correctamente usando un conjunto determinado de **casos de prueba** que reflejan el uso esperado de dicho sistema.

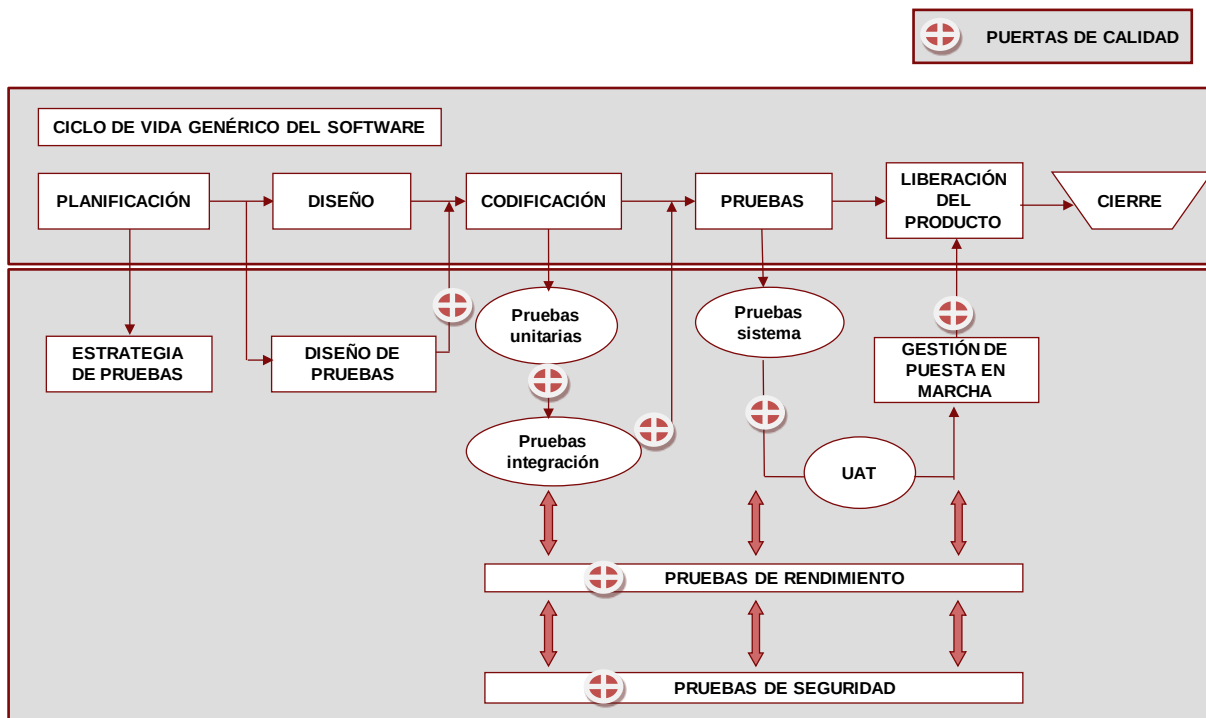
Un caso de prueba es un conjunto de entradas, condiciones de ejecución y resultados esperados, desarrollado para conseguir un objetivo particular o condición de prueba como, por ejemplo, verificar el cumplimiento de un requisito específico. Para llevar a cabo un caso de prueba es necesario definir las precondiciones y post condiciones, identificar unos valores de entrada, y conocer el comportamiento que debería tener el sistema ante dichos valores. Tras realizar ese análisis e introducir dichos datos en el sistema, se observará si su comportamiento es el previsto o no y por qué. De esta forma se determinará si el sistema ha pasado o no la prueba. De ahí su importancia durante la ejecución de pruebas.

Otro dato a considerar es que las pruebas no pueden demostrar que el software está libre de defectos o que se comportará en todo momento como está especificado. Siempre es posible que una prueba que se haya pasado por alto pueda descubrir problemas adicionales con el sistema. *“Las pruebas sólo pueden demostrar la presencia de errores, no su ausencia”*, ([6] Edsger Dijkstra et al., 1972). Las pruebas exhaustivas, en las que cada posible secuencia de ejecución del programa es probada, son imposibles.

Las pruebas, por lo tanto, tienen que basarse en un subconjunto de posibles casos de prueba. Idealmente, algunas compañías deberían tener **políticas** para elegir este subconjunto en lugar de dejar esta tarea al equipo de desarrollo. Estas políticas podrían basarse en políticas generales de pruebas, tal como una política en la que todas las sentencias de los programas deberían ejecutarse al menos una vez. De forma alternativa, las políticas de pruebas pueden basarse en la experiencia de uso del sistema y pueden centrarse en probar características del sistema operacional.

Como parte del proceso de validación y verificación, se debería tomar decisiones sobre quién debería ser responsable de las diferentes etapas de las pruebas. Las pruebas de software se integran dentro de las diferentes fases del ciclo del software dentro de la Ingeniería de Software como se puede observar en la siguiente figura.

Figura 6. Proceso de pruebas en el ciclo de vida de un producto



Durante la etapa de planificación es importante establecer una buena estrategia de pruebas y seleccionar las técnicas adecuadas de estimación en función de los factores que afecten a las pruebas del proyecto. La siguiente fase de desarrollo es el diseño del producto, que trae consigo el diseño de casos de prueba. Durante las siguientes fases de codificación y pruebas del producto se ejecutan las pruebas unitarias, de sistemas, de integración, etc., de las que se hablará en apartados siguientes.

Nunca se debe probar el software en un entorno de producción. Es necesario probar los nuevos sistemas en un entorno de pruebas separado físicamente del de producción. Para crear un entorno de pruebas en una máquina independiente de la máquina de producción es necesario crear las mismas condiciones que hay en la de producción. Existen a tal efecto distintas herramientas que pueden ayudar en la ejecución de pruebas, por ejemplo, herramientas que reproducen automáticamente las bases de datos para simular un entorno de producción.

Por lo tanto las pruebas pueden considerarse como un **proceso** que intenta proporcionar confianza en el software y cuyo objetivo fundamental es demostrar al desarrollador y al cliente que el software satisface sus requisitos.

3.2.1.1. Proceso de pruebas

El proceso de pruebas puede considerarse como un subproyecto dentro del proyecto sobre el cual se están ejecutando las pruebas, y como tal requiere la definición de un plan a seguir. Cuando el proceso de pruebas existe dentro del contexto del proyecto, debería

prestarse atención a la efectividad y eficiencia de las pruebas desde la perspectiva del proyecto y no desde la perspectiva del propio sub proyecto de pruebas.

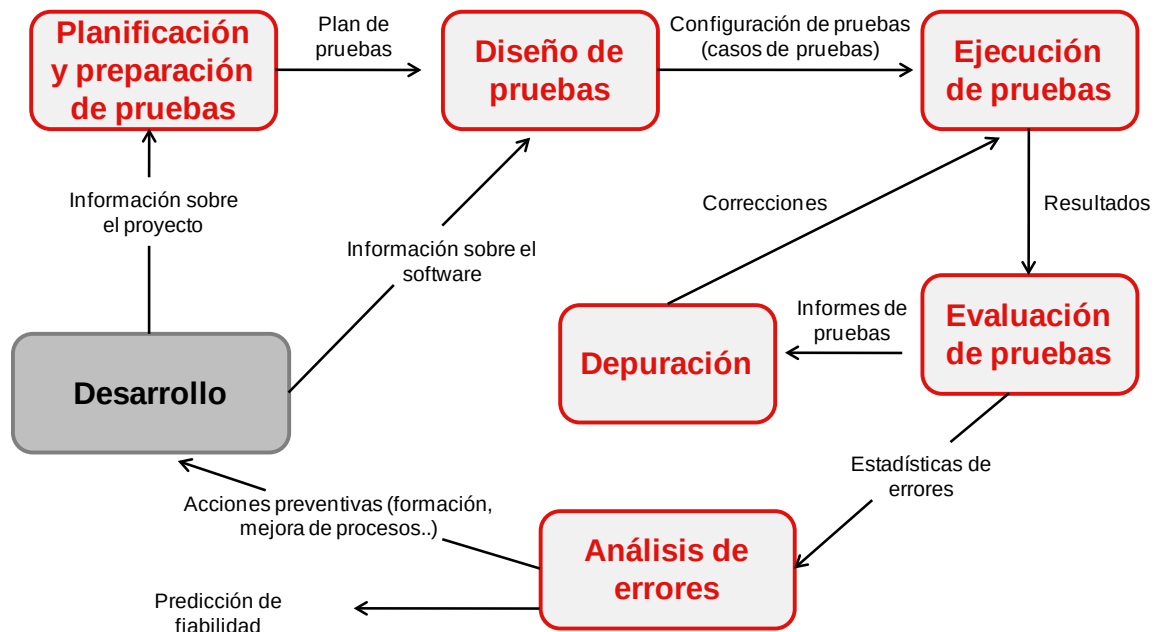
La eficiencia consiste en conseguir el efecto deseado de la manera correcta, es decir, sin desaprovechamiento de recursos, ni de tiempo ni de dinero. Es decir, la eficiencia está relacionada con dos conceptos: productividad y ausencia de pérdidas. Para conseguir esta eficiencia deseada durante el proceso de pruebas se pueden considerar los siguientes aspectos:

- **Evitar redundancias:** las redundancias traen consigo una pérdida o desaprovechamiento de tiempo por lo que hay que intentar ejecutar las pruebas más adecuadas a cada situación y lo más rápido posible. Es importante encontrar los errores que más impacto puedan tener sobre el producto lo más rápido posible. Aunque sea aconsejable no desaprovechar el tiempo, no hay que olvidarse de la planificación, preparación de las pruebas, y de prestar atención a todos los detalles. No abandonar las estrategias previamente establecidas ante la primera señal de problemas o retrasos. Es decir, en un intento de ahorrar tiempo, se debe tener cuidado de no cometer errores que tendrán como consecuencias invertir más tiempo del que se intenta ahorrar.
- **Reducir costes:** Para reducir costes se debería prestar especial atención a la adquisición de elementos que puedan ayudar a la ejecución de pruebas, del tipo de herramientas para la ejecución de pruebas o entornos de pruebas. Habría que cerciorarse de que realmente fueran necesarias y de que existe el tiempo y las habilidades suficientes para emplearlas de manera ventajosa. También es aconsejable evaluar las herramientas antes de comprarlas y ser capaz de justificar estos gastos frente al análisis de costes-beneficios.

Este proceso de pruebas engloba las actividades de definición, elaboración, ejecución y evaluación de pruebas del software. Adicionalmente, se contemplan también el mecanismo de comunicación y resolución de casos fallidos producidos durante la ejecución de las pruebas, así como la elaboración de la documentación de usuario (manuales de usuario y de administración). Si se detectasen errores en los manuales, éstos deberían ser corregidos mediante el proceso de verificación “revisión entre pares” anteriormente definido.

A continuación se muestra un gráfico que muestra un posible flujo del proceso de pruebas, identificando distintas actividades y entregables:

Figura 7. Flujo del Proceso de pruebas



Tras la superación de todas las actividades del proceso de pruebas del software se dispondrá de las siguientes salidas:

- Producto probado y listo para su implantación.
- Planes de prueba con los casos de prueba identificados.
- Informes de pruebas, con los resultados obtenidos de las pruebas, los errores detectados, las acciones llevadas a cabo para su corrección y la evidencia final de superación de todas las pruebas.
- Elementos de prueba, con distintos scripts, programas de prueba, datos de prueba,..., desarrollados para la ejecución y reproducción de las pruebas.

Durante el proceso de pruebas es importante cubrir los siguientes **objetivos**:

- Establecer la participación de los roles implicados.
- Definir el alcance, momento y características de las diferentes pruebas a realizar.
- Definir los contenidos de los manuales de usuario y de administración.
- Establecer los requisitos necesarios para la aceptación del producto antes de su promoción a la siguiente fase del ciclo de vida de desarrollo.
- Definir el ciclo de vida de gestión de un caso de prueba.

- Establecer los mecanismos de resolución de casos fallidos producidos en las pruebas del software.
- Presentar técnicas y estrategias aplicables en la elaboración y ejecución de las pruebas del software.

Además de los objetivos anteriormente expuestos, quizás uno de los más importantes es “definir el contenido de un **plan de pruebas** y de un **informe de pruebas**”. Para finalizar con este apartado dedicado al proceso de pruebas, se van a definir estos dos términos clave en el proceso de pruebas.

El **plan de pruebas** es un documento que describe el alcance, enfoque, recursos y planificación de las actividades de prueba. El plan identifica, entre otros elementos de pruebas, características a probar, tareas de pruebas, quién realizará cada tarea, el entorno de pruebas, riesgos requeridos en el plan de contingencia, técnicas de diseño de pruebas o criterios de entrada y salida a utilizar. Los planes de pruebas, además de ayudar a los gestores a asignar recursos y a estimar el calendario de las pruebas, son de utilidad para los ingenieros software implicados en el diseño y la realización de las pruebas del sistema. Los planes de prueba ayudan al personal técnico a obtener una panorámica general de las pruebas del sistema y a ubicar su propio trabajo en este contexto.

Y por otra parte tendríamos el **informe de pruebas**, que también son una parte muy importante del proceso de pruebas. Estos informes son documentos que recogen información del tipo: objetivo y alcance de las pruebas, resultados obtenidos durante la realización de pruebas, evaluaciones de los elementos de pruebas respecto a sus correspondientes criterios de salida, resultado final de las pruebas...

Tomando como punto de partida el plan de pruebas, en el informe de pruebas se reflejarán los resultados de la ejecución de los casos de prueba definidos, debiendo reflejarse las distintas ejecuciones realizadas sobre un caso de prueba, hasta la superación del mismo.

Estos informes de pruebas son desarrollados por las personas que hayan realizado las pruebas (técnicos de pruebas). Los técnicos de pruebas utilizarán los informes de pruebas para comunicar al resto de personas involucradas en el negocio los resultados de las mismas y de esta manera se podrán tomar decisiones adecuadas.

3.2.1.2. Sistema de pruebas

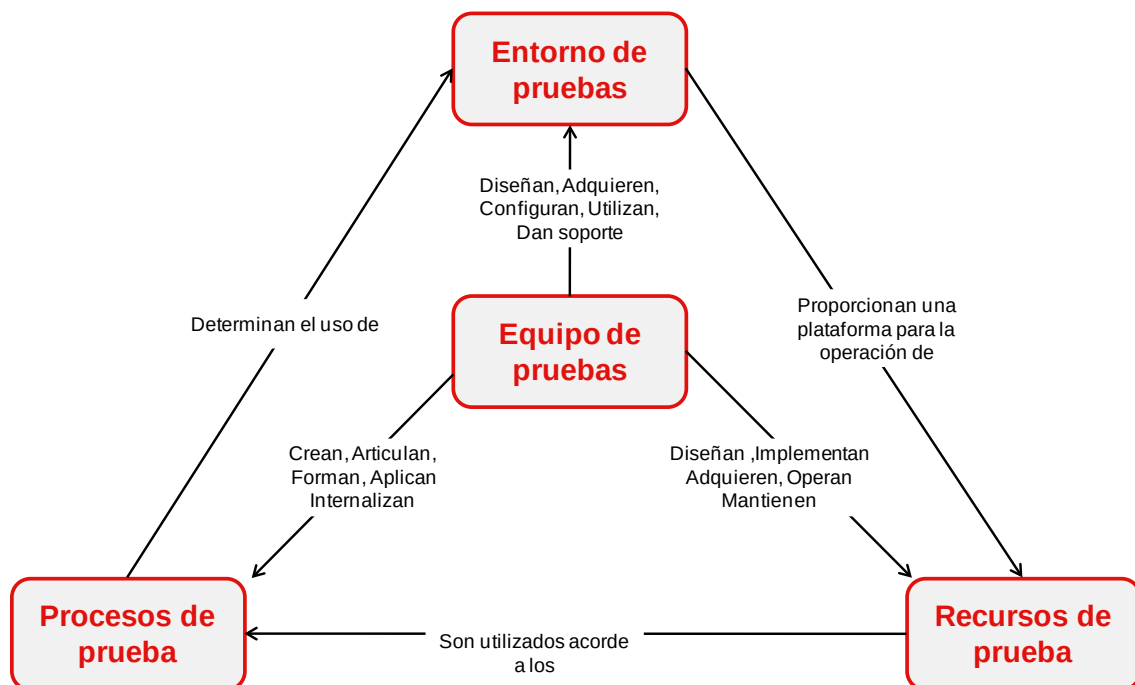
Las pruebas forman parte de lo que se denomina sistema de pruebas. Los cuatro componentes principales de un sistema de pruebas son:

- **Equipo de pruebas:** los ingenieros de pruebas, técnicos de pruebas y el responsable de las pruebas, los cuales tienen habilidades, experiencia y trabajan para diseñar, implementar, y usar componentes de pruebas.
- **Recursos de prueba:** casos de prueba, datos de prueba, herramientas de pruebas, y otro material de desarrollo.

- **Procesos de prueba:** condiciones informales, formales, no documentadas y documentadas en las que se realiza el trabajo de pruebas.
- **Entorno de pruebas:** hardware, software, infraestructura de redes, oficina y laboratorio, y otros elementos que formen el lugar de trabajo.

Para ser efectivos y eficientes, los técnicos de pruebas necesitan el sistema de pruebas correcto. El mejor equipo de pruebas podrá conseguir resultados mediocres utilizando malas herramientas. La relación entre los cuatro componentes del sistema de pruebas aparece plasmada en la siguiente figura:

Figura 8. Relación componentes sistema de pruebas



La **arquitectura del sistema de pruebas** incluye los principios de diseño, la estructura y las herramientas elegidas con las que se construirá el sistema. Es decir, la arquitectura del sistema de pruebas debería reflejar el sistema bajo pruebas. La **ingeniería del sistema de pruebas** es la implementación de la arquitectura del sistema de pruebas, e implica el desarrollo del sistema de pruebas organizado y estructurado. Mediante una buena arquitectura e ingeniería del sistema de pruebas, el equipo de pruebas traducirá las estrategias y tácticas de pruebas en un sistema de pruebas consistente.

Se puede medir la calidad de un sistema de pruebas al igual que se puede medir la calidad del sistema bajo pruebas. Por ejemplo, se podría utilizar la estructura de características de calidad propuesta por la ISO 9126:

- El sistema de pruebas debe ser **funcional**. Debería cubrir los riesgos de calidad críticos.

- El sistema de pruebas debe ser **fiable**. Cuando se ejecuta la misma prueba sobre el mismo sistema bajo pruebas, deberían obtenerse los mismos resultados. El sistema de pruebas debe ser robusto. Cambios pequeños y errores menores en el entorno de prueba no deberían causar mayores fallos en las pruebas. Además, debería poder usarse el sistema de pruebas de manera flexible, pudiendo ejecutar pruebas en distinto orden.
- Un sistema de pruebas debe ser útil para todos aquellos que quieran **usarlo** (técnicos de pruebas u otros usuarios). La curva de aprendizaje del sistema de pruebas debería ser corta para personal cualificado. Para los sistemas de pruebas automatizados, el sistema de pruebas debería capturar sus resultados de manera consistente en cuanto al formato y estructura de los archivos de registro.
- Un sistema de pruebas debe ser **portable**, es decir, debería permitir a los técnicos de prueba ejecutar pruebas en cualquier plataforma.
- Un sistema de pruebas debe ser **eficiente**. La ejecución de pruebas debería ser rápida.
- Un sistema de pruebas ha de poder **mantenerse** con facilidad, siendo capaz de dar soporte a nuevas plataformas y características.

3.2.2. Estrategia de pruebas

Como no es rentable ni, en ocasiones, viable, detectar y corregir todos los fallos existentes en un sistema, es importante encontrar un equilibrio entre una tasa de defectos aceptable y la inversión necesaria para conseguirlo. Hay que analizar el riesgo del negocio y, en función de dicho riesgo, tomar las decisiones para optimizar las pruebas. Para conseguir esta optimización es necesario definir una buena estrategia de pruebas, es decir, definir una serie de principios e ideas que puedan ayudar a guiar las actividades de pruebas.

Existen distintas estrategias de pruebas, y dependiendo de su alineamiento con el objetivo de las pruebas y con los propios proyectos, estas estrategias pueden tener éxito o fallar. Otro punto a tener en cuenta es que no tiene por qué elegirse una sola estrategia. Puede utilizarse una estrategia de manera dominante y utilizar otras de complemento.

Los tipos de estrategia de pruebas más importantes son:

3.2.2.1. Estrategia analítica

Las estrategias de pruebas analíticas tienen en común el uso de alguna técnica analítica formal o informal normalmente durante la etapa de gestión de requisitos y de diseño del proyecto. Por ejemplo:

- **Estrategia orientada a objetos.** Para determinar el enfoque de las pruebas se presta atención a los requisitos, el diseño, y la implementación de objetos. Estos objetos pueden incluir especificaciones de requisitos, especificaciones de diseño,

diagramas UML, casos de uso, código fuente, esquemas de base de datos y diagramas entidad-relación. Este enfoque se basa en una buena documentación, por lo que la ausencia de la misma implica no poder utilizarla.

- **Estrategia basada en riesgos.** Consiste en identificar riesgos estudiando el sistema, documentación de proyectos anteriores, objetivos de la configuración y cualquier otro dato que pueda encontrarse o que pueda aportar la gente involucrada en el proyecto. El diseño, desarrollo y ejecución de pruebas basadas en el conocimiento previo puede ser beneficioso. Este enfoque es apropiado si se dispone de tiempo para investigar el sistema.

Los elementos que están siendo analizados a menudo se denominan “base de las pruebas”. Los resultados del análisis guían el esfuerzo de pruebas, a menudo a través de algunas formas de análisis de cobertura durante el diseño, desarrollo de la ejecución y obtención de resultados de las pruebas. Estas estrategias tienden a ser minuciosas y buenas a la hora de mitigar riesgos de calidad y encontrar errores. Sin embargo, se requiere una inversión de tiempo importante.

3.2.2.2. Estrategia basada en modelo

Las estrategias de pruebas basadas en modelo desarrollan modelos que representan cómo el sistema debería comportarse o trabajar. Las estrategias de pruebas basadas en modelo tienen en común la creación o selección de algún modelo formal o informal para simular comportamientos de sistemas críticos, normalmente durante las etapas de requisitos y diseño del proyecto. Existen distintos tipos de estrategias basadas en modelos:

- Con una **estrategia basada en escenario** se realizan pruebas acorde a escenarios del mundo real. Estos escenarios deberían abarcar la funcionalidad del sistema. En el mundo orientado a objetos, se podría usar una estrategia basada en casos de uso, donde se confíe en documentos de diseño orientados a objetos conocidos como casos de uso. Estos casos de uso son modelos de cómo el usuario, clientes y otras partes involucradas en el negocio utilizan el sistema y cómo debería trabajar bajo estas condiciones.
- Con una **estrategia basada en el dominio** se pueden analizar diferentes dominios de datos de entrada aceptados por el sistema, procesamiento de datos del sistema, y datos de salida entregados por el sistema. Se decidirá cuál será el mejor caso de pruebas para cada dominio, se determinará la probabilidad de errores, frecuencia de uso y entornos desplegados.
- Con una **estrategia basada en un modelo**, se diseñan, desarrollan y ejecutan las pruebas que cubran los modelos que se hayan construido. Esta estrategia será útil dependiendo de la capacidad del modelo para capturar los aspectos esenciales o potencialmente problemáticos del sistema.

3.2.2.3. Estrategia metódica

La estrategia metódica tiende a seguir una línea relativamente informal pero con un enfoque ordenado y predecible que ayuda a comprender dónde probar.

- **Estrategia basada en el aprendizaje.** Se utilizan listas de control que se han desarrollado para guiar el proceso de pruebas. Para desarrollar estas listas de control puede ser de ayuda el basarse en los errores que se han encontrado previamente, en lecciones aprendidas de otros proyectos y en cualquier otra fuente.
- **Estrategia basada en funciones.** Se identifica y se prueba cada función del sistema por separado. Lo mismo ocurre con la **estrategia basada en estados**, donde se identifica y prueba cada estado y cada posible transición de estados que pueda ocurrir.
- **Estrategia basada en la calidad:** Se utiliza una jerarquía de calidad, como la que ofrece la ISO 9126, para identificar y probar la importancia de cada una de las características de la jerarquía como, por ejemplo, la usabilidad, el rendimiento, la funcionalidad o la escalabilidad.

Con una estrategia de pruebas metódica, se siguen estos estándares como objetivos de pruebas. Estas estrategias pueden ser rápidas y efectivas en contra de sistemas que permanecen relativamente estables, o sistemas que son similares a otros ya probados. Los cambios significativos pueden frenar temporalmente estas estrategias hasta que se puedan volver a ajustar los objetivos de las pruebas a la nueva situación del sistema.

3.2.2.4. Proceso o estándar conformista

Las estrategias de pruebas orientadas al proceso llevan el enfoque metódico un paso más allá a la hora de regular el proceso de pruebas. Estas estrategias siguen un desarrollo externo orientado a pruebas a menudo con pocas posibilidades de personalización. Un ejemplo de este tipo de estrategias es la **estrategia de pruebas estandarizada**, se sigue un estándar oficial y reconocido, por ejemplo el estándar IEEE 829 orientado a la documentación de las pruebas. Este estándar, por ejemplo, es utilizado en algunas organizaciones para asegurar la regularidad y completitud de todos los documentos de pruebas. Esta estandarización puede ayudar a hacer que el proceso de pruebas sea más transparente y comprensible para los programadores, responsables, analista de negocio y otras personas ajenas a las pruebas.

3.2.2.5. Estrategia dinámica

Las estrategias de pruebas dinámicas, como las estrategias de pruebas ágiles, minimizan la planificación por adelantado y prueban el diseño. Adaptan todo lo posible el sistema bajo prueba a las condiciones que habrá cuando se libere. Típicamente, enfatizan las últimas etapas de pruebas. Se trata de crear un pequeño conjunto de pautas de pruebas que se enfoquen en debilidades conocidas del software.

- Con una **estrategia de pruebas intuitiva** se puede probar acorde a la experiencia e instinto del equipo de pruebas.
- Con una **estrategia de pruebas exploratoria** se puede aprender simultáneamente sobre el comportamiento y diseño de pruebas, a la vez que las pruebas se van ejecutando y se van encontrando errores. Se irá refinando el enfoque de las pruebas en función de los resultados obtenidos de las pruebas.

Las estrategias de pruebas dinámicas valoran la flexibilidad y la facilidad de encontrar errores. Estas estrategias no producen buena información normalmente acerca de cobertura, mitigación sistemática de riesgos ni ofrecen la oportunidad de detectar defectos en las primeras fases del ciclo de vida del producto. Obviamente, aplicar estas estrategias es mejor que no realizar ningún tipo de pruebas y, cuando van unidas a estrategias analíticas, pueden servir como una excelente herramienta para identificar el vacío dejado por las mismas.

3.2.2.6. Estrategia de pruebas filosófica

Las estrategias de pruebas filosóficas comienzan con una filosofía o creencia de las pruebas.

- Cuando se usa una **estrategia de pruebas exhaustiva** se asume que todo puede tener errores. Es decir, se decide que la posibilidad de que haya fallos latentes es inaceptable por lo que se soportará un considerable esfuerzo al encontrar todos los errores.
- Con la **estrategia de pruebas shotgun** se asume que todo y nada puede tener y tendrá errores. Sin embargo, en este caso, se acepta que no se puede probar todo. Cuando se llegue al punto de no tener una idea sólida acerca de por dónde empezar a probar, se intentará distribuir de manera aleatoria el esfuerzo de pruebas teniendo en cuenta las restricciones de recursos y la agenda.
- Con una **estrategia guiada externamente** no sólo se acepta que no se puede probar todo, sino que también se asume que no se sabe donde están los errores. Sin embargo, se confía en que otras personas puedan tener una buena idea sobre donde están. Para ello, se pedirá ayuda a estas personas sobre la decisión a tomar acerca de si los resultados obtenidos son o no correctos. Por ejemplo, se puede preguntar a los usuarios, personas que den soporte técnico, analistas de negocio o desarrolladores del sistema sobre qué probar o incluso confiar en ellos para hacer las pruebas. Enfatizan las últimas etapas de pruebas simplemente debido a la falta de reconocimiento del valor de pruebas tempranas.

3.2.2.7. Regresión

La regresión es la mala conducta de una función, atributo o característica previamente correctos. La palabra regresión también se suele usar para definir el descubrimiento de un error que previamente no se encontró durante la ejecución de una prueba. La regresión

normalmente está asociada a algún cambio en el sistema, como añadir una característica o arreglar un error.

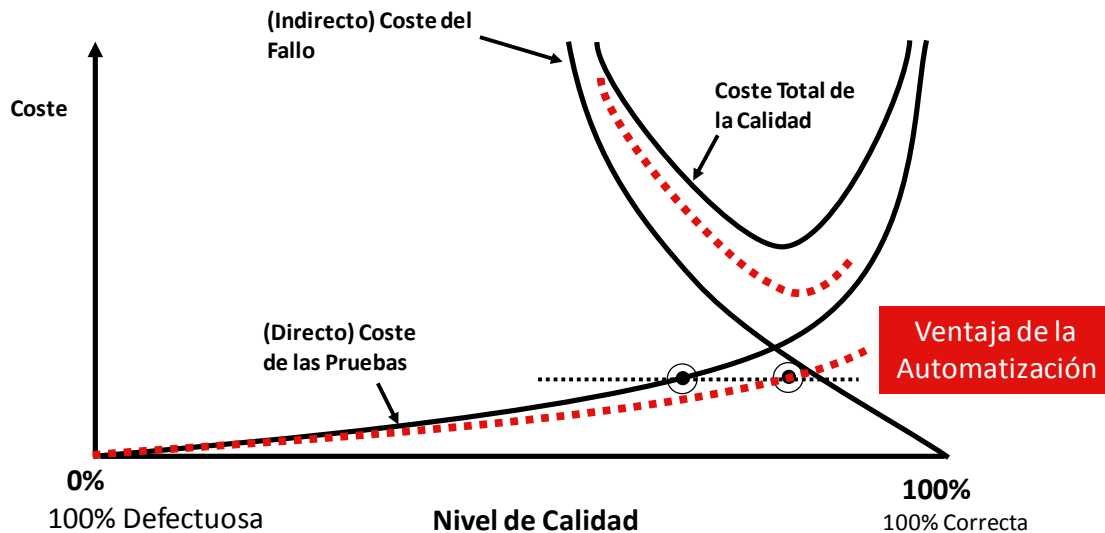
La regresión puede ser de tres tipos:

- **Regresión local:** al producirse un cambio o arreglarse un error se crea un nuevo error.
- **Regresión de exposición:** al producirse un cambio o arreglarse un error se identifican errores ya existentes.
- **Regresión remota:** un cambio o el arreglo de un error en un determinado área produce un fallo en otro área del sistema. La regresión remota es la regresión más difícil de detectar, ya que los usuarios, clientes y el resto de los interesados en el proyecto tienden a confiar en las características ya existentes del sistema. Por ello, los técnicos de pruebas deberían tener estrategias que sean efectivas en contra de todas las causas y efectos de las regresiones.

Una posible estrategia para enfrentarse a la regresión, quizás el enfoque más simple, es la **fuerza bruta**. Para la mitigación del riesgo de regresión, la estrategia de la fuerza bruta consiste en repetir todas las pruebas. Imaginemos que se ha desarrollado un conjunto de pruebas bien alineadas con la calidad. En este caso, se habrá desarrollado un análisis de riesgos de calidad sólido y se tendrá tiempo y recursos suficientes para cubrir todos los riesgos críticos de calidad. Si se repiten todas las pruebas después del último cambio realizado, se deberían encontrar todos los errores de regresión importantes.

- La **automatización**. Se puede considerar la automatización como una estrategia para aumentar la calidad de un producto a un bajo coste y optimizar el esfuerzo de las pruebas, como se ve reflejado en la siguiente figura:

Figura 9. La automatización como parte de la estrategia de pruebas



Fuente: J.M. Juran's Quality Control Handbook
Giga Information Group 2001, Justifying IT Investments: Quality Assurance

La automatización es la única manera de repetir todas las pruebas sobre sistemas complejos y grandes. La automatización es práctica cuando los costes del diseño e implementación de pruebas automatizadas sean recuperables, y se vayan a ejecutar de manera frecuente.

A pesar de su gran utilidad, en ocasiones, la inversión extra que supone para el proyecto y la necesidad de ciertas habilidades por parte de miembros de la organización hacen que la automatización sea un proceso costoso. Sin embargo, algunas de sus ventajas son:

- La automatización puede reducir drásticamente el esfuerzo de las pruebas de regresión.
- La automatización permite realizar validaciones durante los ciclos de cambios, cosa que sería imposible hacer manualmente debido a restricciones de tiempo.
- La automatización habilita la consistencia y cobertura lógica. No hay riesgo alguno de excluir casos de prueba o pasar por alto errores si se diseña correctamente.

Esta estrategia suele envolver pruebas funcionales automatizadas antes de la liberación del producto, pero algunas veces, se centra por completo en funciones liberadas con anterioridad. Por ejemplo, se puede intentar automatizar todas las pruebas de sistemas de tal forma que cuando se produzca cualquier cambio se pueda volver a ejecutar cada prueba para asegurarse de que ningún área se ha visto afectada. Pero, incluso con una automatización efectiva, no siempre es posible repetir todas las pruebas ya que no resulta

práctico o su gestión es insostenible. Por ello, en ocasiones se necesitará realizar una selección sobre el conjunto de pruebas a automatizar.

A continuación se describen algunas técnicas que pueden ayudar a realizar esta selección:

- Uso de **trazabilidad**, donde las pruebas estén relacionadas con el comportamiento del sistema como, por ejemplo, con elementos de la especificación de requisitos, elementos de la especificación del diseño o riesgos relacionados con la calidad. Por lo tanto, se deberá prestar atención a si estos elementos se ven afectados por algún cambio o por el arreglo de errores, realizar un seguimiento de las pruebas asociadas y seleccionar las pruebas que se van a volver a ejecutar.
- Uso de **análisis de cambios**. En este caso se presta atención a las descripciones estructurales del sistema, definiendo cómo los efectos de los cambios afectan al sistema. A menos que se tenga un profundo entendimiento sobre el diseño y programación del sistema, probablemente será necesaria la ayuda de personas especializadas en el tema.
- Uso de **análisis de riesgos relacionados con la calidad**. Con la trazabilidad y el análisis de cambios se están utilizando riesgos técnicos para decidir qué volver a probar. Sin embargo, se debería también revisar el análisis de riesgos de calidad para determinar qué áreas tienen un alto riesgo que pueda afectar al negocio.

3.2.3. Niveles de pruebas

No conseguir que las pruebas se adecúen al proyecto es tan peligroso como probar los atributos equivocados del sistema o usar estrategias de pruebas incorrectas.

Las organizaciones tienen distintas necesidades y buscan distintos beneficios por los que realizar pruebas como pueden ser:

- Mejorar la calidad.
- Disminuir los costes de mantenimiento post-producción.
- Suavizar los ciclos de liberación del producto.
- Cumplir con la legalidad.
- Reducir los riesgos relacionados con el incumplimiento de tareas.

Los técnicos de pruebas normalmente no piensan en estos términos sino que se centran en buscar fallos e intentar saber qué funciona y qué no funciona, dentro de los parámetros definidos en las pruebas. Son los responsables del proyecto quienes piensan desde un enfoque más estratégico y a largo plazo. Por lo tanto, cuando los técnicos de pruebas comunican a dichos responsables información relacionada con las pruebas, lo harán con este enfoque, es decir, describirán de qué manera las pruebas afectarán a la gestión de los

riesgos del proyecto. Los técnicos pueden tratar temas relacionados con la manera en que las pruebas reducen la probabilidad de enfrentarse a costes futuros no esperados. Es importante que exista esta comunicación ya que es la manera de que el proyecto funcione. Los técnicos de pruebas comunican a los responsables la situación del proyecto y ellos autorizan el presupuesto necesario para solventar los problemas.

Una manera de adecuar las actividades de pruebas al proyecto es dividir el esfuerzo total de pruebas en una **secuencia de fases o niveles**. Estos niveles a menudo se organizan por la secuencia en la que las porciones del sistema llegan a estar listas para probar. Cada fase por lo tanto cubre un objetivo de prueba a alto nivel.

- **Pruebas unitarias, de componente o de subsistema.** Se prueban porciones del sistema mientras son creadas, se buscan errores de cada pieza del sistema bajo pruebas antes de que se produzca la integración de las mismas.
- **Pruebas de integración.** Se prueba una colección de unidades, subsistemas o componentes que se están comunicando o interactuando, buscando errores en las relaciones y conexiones entre pares y grupos de estas piezas del sistema bajo prueba.
- **Pruebas de sistema.** Se prueba el sistema entero, buscando errores en el comportamiento, funciones y respuestas particulares y globales del sistema bajo prueba como un todo.
- **Pruebas de aceptación o piloto.** No buscan errores o fallos. El objetivo es demostrar que el producto está listo para el paso a producción.

Definitivamente, el tener un entendimiento claro y una definición minuciosa de los niveles de pruebas ayudará a reconocer áreas no identificadas y a prevenir repeticiones y solapamientos en las pruebas, aunque también es cierto, que en ocasiones se introducen solapamientos deliberadamente para tratar riesgos específicos.

Estos niveles de pruebas forman parte de los procesos de validación o verificación, alguno de ellos incluso forma parte de ambos como, por ejemplo, las pruebas unitarias que se realizan durante la fase de codificación. Estas pruebas forman parte de la validación en el sentido de que detectan defectos y ayudan a conseguir el producto correcto. También están relacionadas con el proceso de verificación, ya que ayudan a comprobar, por ejemplo, que se están siguiendo ciertos estándares de codificación, consiguiendo código reutilizable, o que se están cumpliendo buenas prácticas de modularidad y encapsulación. Por otro lado están las pruebas de integración y de sistemas, que entrarían en las actividades de verificación ya que ayudan a asegurar que el producto está de acuerdo con su especificación y con sus requisitos, tanto funcionales como no funcionales. Y por último estarían las pruebas de aceptación que sólo entrarían en el proceso de validación, ya que ayudan a comprobar si el usuario acepta el producto, es decir, si satisface sus necesidades

Se va a describir en más profundidad cada uno de estos niveles en apartados sucesivos.

3.2.3.1. Pruebas unitarias

Incluidas dentro de la fase de desarrollo del producto, las pruebas unitarias se realizan para cada módulo individual del sistema, también denominados unidades o componentes, antes de proceder a su integración.

Las pruebas unitarias intentan asegurar que cada unidad cumple con las especificaciones antes de integrarlas con otras unidades. Además, comprueban que el código de cada unidad puede ser ejecutado sin problemas.

Las pruebas unitarias pueden realizarse de forma aislada al sistema mediante el uso de stubs y drivers que reemplazan el software omitido o ausente y simulan la interconexión entre los componentes del software de una forma simple.

Estas pruebas son realizadas por el propio desarrollador a la par que el diseño y construcción del módulo.

En la medida de lo posible, se abogará por la automatización y repetitividad de las pruebas unitarias, así como por la posibilidad de realizar pruebas de regresión sobre las mismas, para lo cual se recomienda el empleo de herramientas de pruebas unitarias.

Tabla 1. E/S y roles en las pruebas unitarias

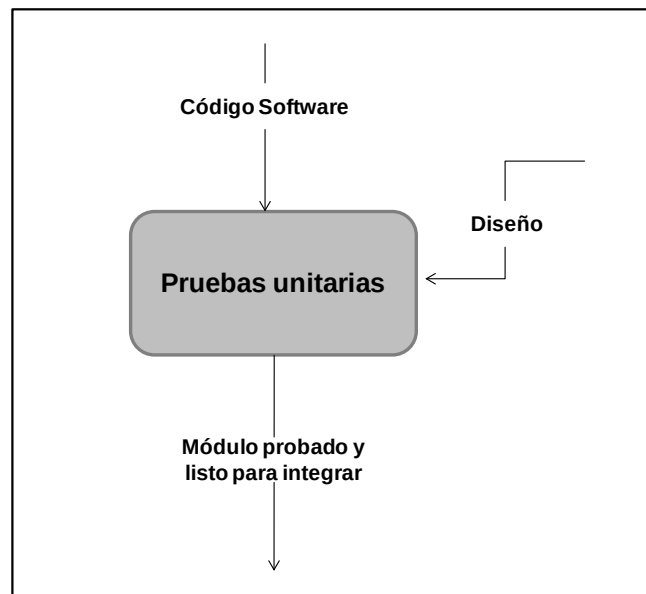
Entradas	Código Software. Diseño Detallado.
Salidas	Módulo Probado y Listo para Integrar.
Roles	Desarrollador.

Las tareas o pasos más relevantes a seguir durante las pruebas unitarias se describen a continuación:

- Mediante inspección visual del código o cualquier otro medio, como puede ser la construcción de pequeños programas auxiliares de prueba, el desarrollador revisará la correcta lógica y funcionamiento del módulo. Se intentará controlar los siguientes elementos:
 - Prueba y control de errores y excepciones.
 - Tipo y valor de los parámetros de entrada y salida en métodos y funciones.
 - Verificación de la lógica de negocio y algorítmica del módulo.
 - Revisión de condiciones y valores límite.

- Prueba de elementos de repetición o bucles, y anidaciones entre los mismos.
- Simulación de caminos y alternativas posibles.
- Evaluación de los resultados obtenidos frente a los resultados esperados.
- Corrección del módulo hasta alcanzar su correcto funcionamiento.

Figura 10. Flujo de control pruebas unitarias



3.2.3.2. Pruebas de integración

Esta fase toma como punto de partida el producto integrado resultado de la fase de desarrollo, y constituyente de la línea base de integración de la gestión de configuración.

Tabla 2. E/S, tareas y roles en las pruebas de integración

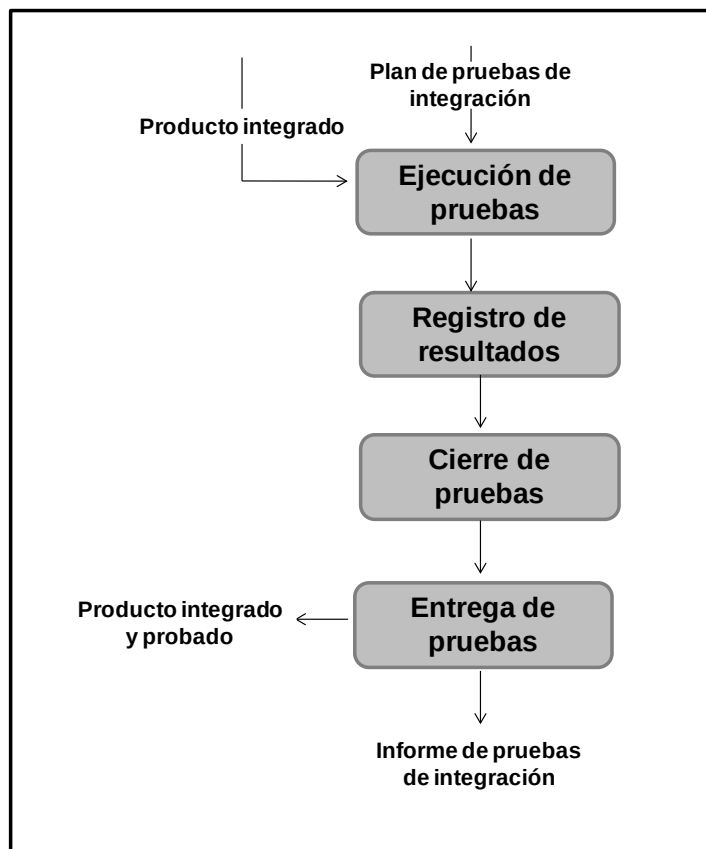
Entradas	Producto integrado. Plan de Pruebas de Integración.
Tareas	Preparación del entorno de pruebas. Instalación del sistema en el entorno de pruebas. Construcción de elementos auxiliares de prueba. Ejecución de las pruebas. Obtención y registro de resultados. Elaboración del Informe de Pruebas, en el que se registrarán los resultados de las pruebas. Corrección de fallos y errores detectados. Reiteración de la actividad hasta superar todas las pruebas. Revisión del Informe de Pruebas, comprobando la correcta ejecución de todas las pruebas planteadas y del contenido del Informe de Pruebas. Cierre formal de la fase antes de la entrega del producto integrado y probado a pruebas. Transferencia a pruebas del producto integrado y probado con el objeto de que sobre el mismo puedan realizarse las pruebas de sistema. Dicha entrega se constituye en la recién creada Línea Base de Sistema.
Salidas	Informe de Pruebas de Integración. Producto listo para su entrega a pruebas.
Roles	Ingeniero de pruebas. Jefe de desarrollo.

Una vez que las unidades se han desarrollado, la siguiente fase será unirlos para crear el sistema. Es decir, habrá que recopilar todos los componentes, ensamblarlo y prepararlos para las pruebas. A partir de este producto integrado, se realizarán las pruebas pertinentes para verificar la correcta integración y cohesión del sistema.

Al realizar estas pruebas se tomarán como referentes las especificaciones del documento de análisis y más especialmente del de diseño, verificando que la división arquitectónica y de

módulos establecida funciona correctamente. Es decir, el propósito principal de las pruebas de integración es descubrir los defectos de las interconexiones y de la interacción entre sistemas o componentes integrados.

Figura 11. Flujo de control pruebas de integración



3.2.3.2.1. Estrategias

Antes de realizar cualquier prueba de integración, es necesario establecer una estrategia y decidir cómo unir las distintas partes de un sistema. En la realización de estas pruebas pueden adoptarse diferentes **estrategias**:

Integración *Big-bang*

Desde el enfoque de esta estrategia de integración, todas las unidades se ensamblan en una sola resultando un sistema completo. Esta técnica tiene la ventaja de que no es necesario simular ninguna parte porque todo está acabado antes de empezar las pruebas de integración. La mayor desventaja es, en general, el tiempo que se consume y la dificultad que supone realizar un seguimiento de las causas de los fallos de esta integración. Cuando se realizan pruebas en este tipo de integración, es realmente difícil conseguir aislar los errores encontrados ya que no se presta atención a los interfaces entre unidades individuales.

Esta estrategia suele ser una mala elección ya que implica el riesgo de descubrir problemas al final del proyecto, por lo que es más caro resolverlos.

Estrategia *bottom-up*

Desde una estrategia *bottom-up*, se empiezan probando los módulos de nivel inferior (más especializados) para, a continuación, ir ascendiendo progresivamente probando los módulos superiores (mayor nivel de abstracción). Dado que no se dispone del sistema completo hasta el final, deberán ir creándose programas auxiliares de nivel superior (*Test-Drivers*) que permitan probar la integración de los componentes del sistema en cada momento.

Características:

- Requiere de la construcción de elementos de prueba (*test-drivers*) en cada nivel.
- No se encuentran problemas de diseño hasta muy avanzado el proceso.
- Es apropiado para sistemas orientados a objetos (OO).
- Es necesario para componentes críticos.

Estrategia *top-down*

En la estrategia *top-down*, partiendo de los subsistemas o componentes de nivel superior (mayor nivel de abstracción), se prueba la integración entre los mismos. En sucesivas iteraciones se va descomponiendo el sistema en subsistemas de más bajo nivel (módulos inferiores más especializados) y se prueba la integración entre ellos. Se han de construir componentes auxiliares que simulen el correcto funcionamiento de módulos de más bajo nivel mientras éstos no se hayan integrado en el sistema objeto de pruebas.

Características:

- Esta estrategia de pruebas es usada conjuntamente al desarrollo *top-down*.
- Descubre rápidamente errores en la arquitectura.

3.2.3.3. Pruebas de sistema

Las pruebas de sistema se ocupan del comportamiento del sistema o del producto como un todo. Incluyen pruebas basadas en riesgos y/o en la especificación de los requisitos, procesos de negocio, casos de uso u otras descripciones de alto nivel del comportamiento del sistema, interacciones con el sistema operativo y recursos del sistema. El comportamiento del sistema está documentado en la especificación funcional. Esta especificación debería contener definiciones tanto de los requisitos funcionales como de los no funcionales del sistema.

Tabla 3. E/S, tareas y roles pruebas de sistema

Entradas	Plan de Pruebas de Sistema. Producto integrado y probado.
Tareas	Preparación del entorno de pruebas. Se recomienda la existencia de un entorno de pruebas específico para la realización de este tipo de pruebas. Instalación en el entorno de pruebas. Construcción de elementos auxiliares de prueba. Ejecución de las pruebas. Obtención y registro de resultados. Elaboración del Informe de Pruebas, en el que se registrarán los resultados de las pruebas. Corrección de fallos y errores detectados. Reiteración de la actividad hasta superar todas las pruebas. Elaboración del Manual de Usuario y de Administración. Actualización y revisión del Plan e Informe de Pruebas.
Salidas	Informe de Pruebas de Sistema. Manual de Usuario. Manual de Administración. Plan e Informe de Pruebas Sistema probado.
Roles	Ingeniero de pruebas Analista Funcional. Jefe de pruebas.

Esta fase del proceso de pruebas toma como punto de partida el producto integrado y probado, almacenado en la línea base de sistema. A partir de la misma, se verificará el comportamiento global del sistema. En este punto cabe citar distintos tipos de pruebas a las que el sistema puede ser sometido para verificar su correcto comportamiento y para validar que se satisfacen los requisitos de cliente:

- Pruebas funcionales, para verificar la correcta funcionalidad del mismo.

- Pruebas de instalación, configuración y carga inicial de datos.
- Pruebas de usabilidad.
- Pruebas de migración de datos.
- Pruebas de prestaciones.
- Pruebas de seguridad.

Los tipos de pruebas concretos a realizar dependerán de cada sistema, indicándose en el plan de pruebas.

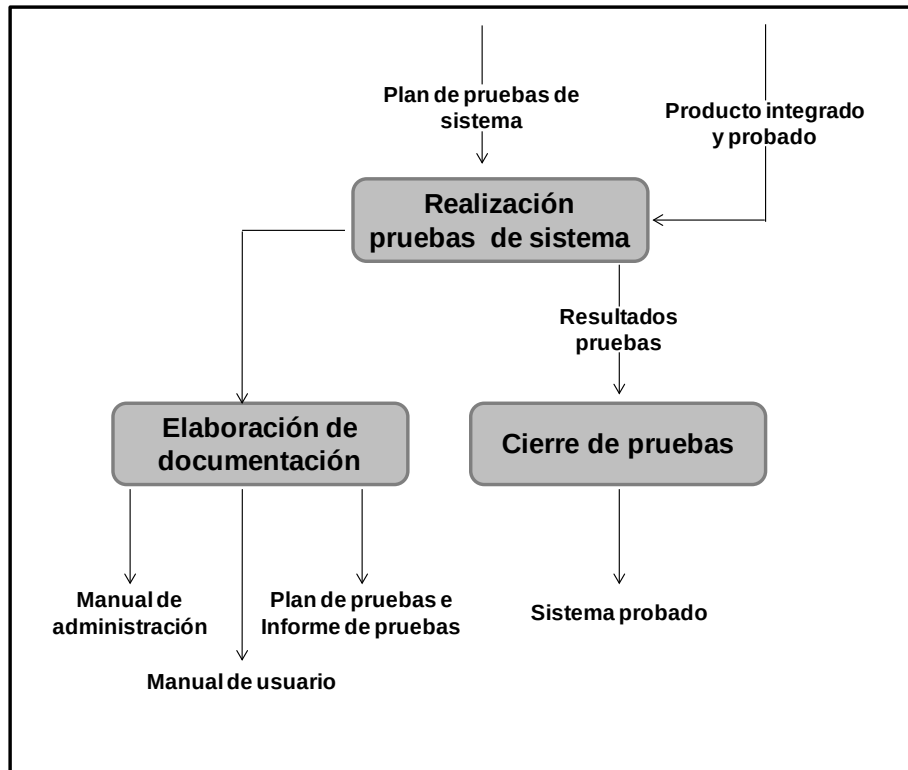
Adicionalmente, en esta fase también se elaborarán los documentos destinados al usuario, como son los manuales de usuario y de administración. La elaboración de estos manuales puede ser realizada en paralelo con el resto de actividades de la fase, sirviendo de ayuda y referente en la confección y ejecución de las pruebas a realizar.

Las pruebas de sistema normalmente son llevadas a cabo por un equipo independiente de técnicos especializados en pruebas. En algunas organizaciones estas pruebas son llevadas a cabo por un equipo externo o por analistas de negocio.

Las pruebas de sistema de los requisitos funcionales usan técnicas basadas en especificación (o de caja negra) sobre el sistema probado. Las técnicas basadas en estructura suelen usarse para valorar la minuciosidad de los elementos de pruebas. Este y otro tipo de técnicas se tratarán más adelante.

Las pruebas de sistema requieren un entorno de pruebas controlado en lo que se refiere a un control de versiones del software o datos de prueba, entre otras cosas. Una prueba del sistema es ejecutada por la organización en un entorno controlado. El entorno de pruebas debería corresponder al objetivo final o entorno de producción tanto como sea posible, con el objetivo de minimizar el riesgo de encontrar fallos específicos del entorno.

Figura 12. Flujo de control pruebas de sistema



3.2.3.4. Pruebas de aceptación

Las pruebas de aceptación tienen como objetivo obtener la aceptación final del cliente antes de la entrega del producto para su paso a producción. Cuando la organización ha realizado las pruebas de sistema y ha corregido la mayoría de sus defectos, el sistema será entregado al usuario o al cliente para que de su aprobación.

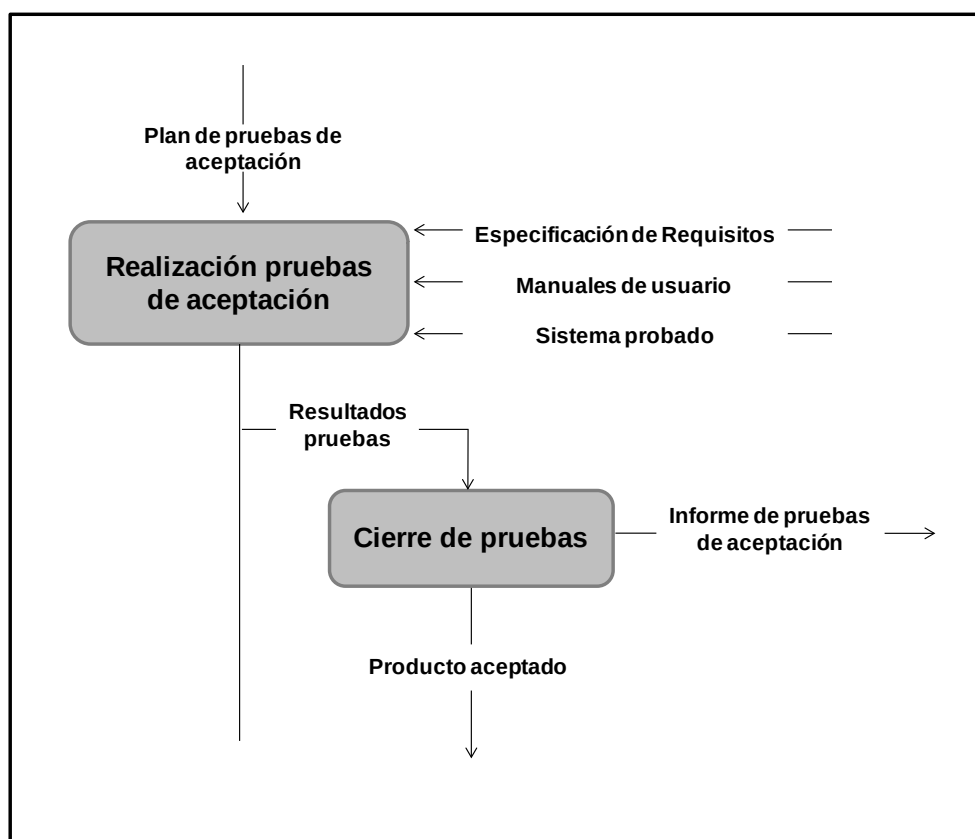
Tabla 4. E/S, tareas y roles de pruebas de aceptación

Entradas	Especificación de Requisitos. Manuales de Usuario. Sistema probado. Plan de Pruebas.
Tareas	Preparación del entorno de pruebas. Se recomienda la existencia de un entorno de pruebas específico para la realización de este tipo de pruebas. Instalación en el entorno de pruebas. Identificación de las pruebas a realizar. Planificación de las pruebas. Se establecerán las posibles dependencias que hubiera entre pruebas y se establecerá el orden o secuencia de ejecución de las pruebas en base a dichas dependencias. Ejecución de las pruebas. Obtención y registro de resultados. Corrección de fallos y errores detectados. Reiteración de la tarea hasta superar todas las pruebas. Elaboración de un Informe de Pruebas de aceptación. Revisión de la correcta ejecución y resultados de todas las pruebas planteadas. Creación de la línea base de producción. Cierre formal de la actividad.
Salidas	Resultados de pruebas. Producto aceptado Informe de Pruebas de aceptación.
Roles	Ingeniero de Pruebas. Jefe de Pruebas. Jefe de Proyecto (Cierre formal de la actividad).

Las pruebas de aceptación, son básicamente pruebas funcionales sobre el sistema completo, y buscan comprobar que se satisfacen los requisitos establecidos. Su ejecución es facultativa del cliente, y en el caso de que no se realicen explícitamente, se dan por incluidas dentro de las pruebas de sistema. Es decir, las pruebas de aceptación son, a menudo, responsabilidad del usuario o del cliente, aunque cualquier persona involucrada en el negocio puede realizarlas. La ejecución de las pruebas de aceptación requiere un entorno de pruebas que represente el entorno de producción.

Esta fase toma como punto de partida la línea base de aceptación del producto ya instalado en el entorno de certificación. A partir de dicha línea base se acepta el producto, tomando como referencia la especificación de requisitos y comprobando que el sistema cubre satisfactoriamente los requisitos del cliente.

Figura 13. Flujo de control pruebas de aceptación



3.2.3.4.1. Estrategias de pruebas de aceptación

Si el sistema ha sido desarrollado para el mercado masivo entonces no será práctico probarlo para usuarios o clientes individuales, en algunos casos sería imposible. En estos casos, antes de que el producto se ponga a la venta, es necesaria una retroalimentación.

A menudo este tipo de sistemas tiene dos etapas de pruebas de aceptación.

Pruebas α

Las pruebas α -alfa consisten en invitar al cliente a que pruebe el sistema en el entorno de desarrollo. Se trabaja en un entorno controlado y el cliente siempre tiene un experto a mano para ayudarle a usar el sistema. El desarrollador va registrando los errores detectados y los problemas de uso.

Pruebas β

Las pruebas β -beta se realizan con posterioridad a las prueba α -alfa, y se desarrollan en el entorno del cliente. En este caso, el cliente se queda a solas con el producto y trata de encontrarle fallos de los que informa al desarrollador.

3.2.4. Buenas prácticas en las pruebas del software

En el proceso de realización de pruebas del software conviene tener presentes e intentar aplicar las siguientes consideraciones y buenas prácticas:

- Se trata de un proceso continuo e iterativo a lo largo del ciclo de vida del software.
- Su principal objetivo es prever y detectar anticipadamente posibles fallos y problemas del software. Adicionalmente, sirve para validar ciertas características del mismo (requisitos y especificaciones técnicas).
- Las pruebas de sistema deben realizarse en un entorno lo más parecido al entorno real de producción del sistema (hardware, software, volumen de datos,...).
- El proceso de pruebas es un proceso ordenado y metódico, repetitivo y sistemático. Se basa en la aplicación de un orden y un método y no en la inspiración y el buen hacer individual del ingeniero de pruebas.
- Deben preverse y planificarse anticipada y ordenadamente las pruebas a las que someter el sistema (casos de prueba). Asimismo, deben gestionarse y documentarse las mismas (resultados previstos, resultados obtenidos, casos fallidos,...) recomendándose la utilización de una herramienta de gestión de pruebas.
- Todo caso de prueba del sistema ha de estar codificado y asociado a uno o varios requisitos para cuya verificación ha sido creado, permitiendo así la trazabilidad entre requisitos y casos de prueba.
- Se recomienda que las pruebas sean repetibles y automatizables, en particular las pruebas de regresión, cuyo propósito es la verificación del correcto funcionamiento del sistema ante cambios en el mismo.

- El entorno de pruebas ha de ser “estable” durante la realización de las mismas. Debe limitarse y controlarse la promoción a dicho entorno de versiones “corregidas” del software. Ante dichas promociones deberían ejecutarse las pertinentes pruebas de regresión.
- Los casos de prueba deben ser concretos, precisos y apropiados. Debe intentar abarcarse el máximo número de posibilidades y cubrir el máximo código posible en las pruebas.
- Se recomienda, siempre que sea posible, que las pruebas sean realizadas por personal con experiencia en la realización de pruebas y no por los propios desarrolladores.
- Los elementos de prueba contruidos deben documentarse convenientemente y guardarse en el repositorio de gestión de configuración junto con el resto de la configuración del proyecto o sistema.

3.2.5. Tipos de pruebas

A continuación, se describe una serie de tipos de pruebas como medio para definir, de forma clara, el objetivo de ciertos niveles de pruebas para un programa o proyecto. Es necesario conocer distintos tipos de pruebas para entender sus objetivos globales. Centrar las pruebas en un objetivo específico y seleccionar el tipo de pruebas apropiado, ayudará a tomar y comunicar decisiones acerca de los objetivos de las pruebas de forma más sencilla.

Cada tipo de pruebas se centra en un objetivo particular. Dependiendo de estos objetivos, una posible clasificación de las pruebas es la siguiente.

3.2.5.1. Pruebas de prestaciones

Dentro de lo que se denomina genéricamente Pruebas de Prestaciones, se pueden englobar distintos tipos de pruebas, con objetivos, estrategias y escenarios muy diferentes pero con el propósito común de medir las prestaciones del sistema. Estas pruebas son:

- **Pruebas de Carga.** Su objetivo es validar los requisitos en cuanto a las prestaciones definidas para el sistema como, por ejemplo: tiempo de respuesta máximo permitido para el número de usuarios contemplado, tratamiento de la concurrencia requerida,... Para su verificación se generan escenarios de carga adecuados, modelando de forma realista el comportamiento de usuarios tipo y recogiendo las métricas adecuadas (tiempo de respuesta, conexiones abiertas,...) según el requisito a validar.
- **Pruebas de Capacidad.** Persiguen la obtención del punto umbral a partir del cual las prestaciones del sistema se degradan, permitiendo ver si el dimensionamiento actual (y futuro) del sistema es adecuado o insuficiente. Para este tipo de pruebas se va incrementando la carga (número de usuarios simulados), recogiendo indicadores que informan del comportamiento del sistema, hasta detectar la saturación del mismo.

- **Pruebas de Estrés.** Su objetivo es verificar el comportamiento del sistema en situación de sobrecarga, es decir, aseguran que el sistema tiene comportamiento adecuado cuando se excede los límites de capacidad de procesamiento y almacenamiento. Especialmente importante es la integridad (funcional, bases de datos,...) en situaciones extremas.
- **Pruebas de Escalabilidad.** Su objetivo es verificar la facilidad y capacidad del sistema para “absorber” requisitos mayores de prestaciones, verificando la rigidez o flexibilidad del sistema para progresar.
- **Pruebas de Estabilidad.** Verifican el comportamiento del sistema en el tiempo, buscando aquellas degradaciones originadas por una incorrecta gestión de los recursos del sistema y una mala liberación de los mismos. En estas pruebas se lanza una carga estable (no creciente) por debajo de la capacidad del sistema y durante un largo periodo de tiempo, verificando que no aumenta la utilización de recursos, ni se degradan las prestaciones.

3.2.5.2. Pruebas de usabilidad

La usabilidad de un sistema software estudia la forma de diseñar las aplicaciones de forma que los usuarios puedan interactuar con ellas de la forma más fácil, cómoda e intuitiva posible, destacando la necesidad de realizar un buen diseño de la interfaz de usuario. Con este propósito, las pruebas de usabilidad comprueban el grado de consecución del mismo dentro de un escenario de trabajo habitual. Estas pruebas y los errores encontrados en las mismas obedecen principalmente a aspectos como:

- La navegación por la aplicación y la secuencia de pasos a seguir para realizar una actividad frecuente.
- La presencia y organización de la información: pantallas, listados,...
- La flexibilidad en la realización de operaciones.
- La existencia de etiquetas y mensajes apropiados.
- La existencia de información de estado: usuario, situación, filtros aplicados,...

Una estrategia muy habitual en este tipo de pruebas es que un conjunto de usuarios realicen un uso asistido de la misma, tomando nota de las dificultades e inconvenientes que se encuentran en la realización de su trabajo.

3.2.5.3. Pruebas de regresión

Son pruebas que buscan descubrir fallos de regresión, esto es, funcionalidades del software que previamente funcionaban correctamente y que ahora no lo hacen de la forma adecuada.

Normalmente estos fallos de regresión ocurren por consecuencias involuntarias debido a cambios de cualquier tipo producidos en el software, que pueden provocar efectos colaterales adversos no deseados.

Los métodos comunes de las pruebas de regresión incluyen re-ejecutar pruebas anteriores y comprobar si el sistema sigue funcionando adecuadamente. Es muy recomendable que estas pruebas se encuentren automatizadas, de forma que en cualquier momento puedan lanzarse para verificar el comportamiento adecuado del sistema.

Muy relacionadas con las pruebas de regresión están las pruebas de confirmación cuyo objetivo es asegurarse de que un defecto corregido lo está realmente. Es decir, cuando una prueba falla, se averigua el defecto del software (la causa del fallo), se informa del defecto encontrado y se espera una nueva versión del software con el defecto arreglado. En este caso será necesario ejecutar de nuevo la prueba para confirmar que el defecto ha sido realmente solucionado. A esta prueba se llama prueba de confirmación. Al realizar una prueba de confirmación es importante asegurarse que es ejecutada exactamente bajo las mismas condiciones y de la misma manera que la primera vez que se llevó a cabo, usando los mismos valores de entrada y el mismo entorno. Si todo sale bien se puede afirmar que el software es correcto para esas condiciones específicas, pero este arreglo puede haber introducido algún defecto distinto en otra parte del software.

3.2.5.4. Pruebas funcionales

Los tipos de pruebas definidas hasta el momento hacen referencia a lo que se denominan pruebas no funcionales, cuyo objetivo es probar la calidad de las características de un producto software, o lo que es lo mismo, los atributos no funcionales del sistema.

Por el contrario, las pruebas funcionales tienen por objetivo probar que los sistemas desarrollados cumplen con las funciones específicas para los que han sido creados. Es común que este tipo de pruebas sean desarrolladas por analistas de pruebas con apoyo de algunos usuarios finales.

La función de un sistema es 'lo que hace' dicho sistema, y normalmente es descrita en una especificación de requisitos, una especificación funcional o en casos de uso. Las pruebas funcionales son pruebas basadas en el análisis de la especificación funcional de un componente o de un sistema.

Las pruebas funcionales, a menudo, se llaman pruebas **de caja negra** porque no enfocan su atención en la manera de generar las respuestas del sistema. El enfoque de este tipo de prueba se basa en el análisis de los datos de entrada y de salida, sin preocuparse del funcionamiento interno del sistema.

El proceso de pruebas para determinar la funcionalidad de un producto software se centra en la conformidad, interoperabilidad, seguridad y en la exactitud. Las pruebas de seguridad por ejemplo, se llevan a cabo para identificar y resolver vulnerabilidades de seguridad antes del despliegue o para identificar de forma periódica y resolver cuestiones de seguridad dentro del sistema.

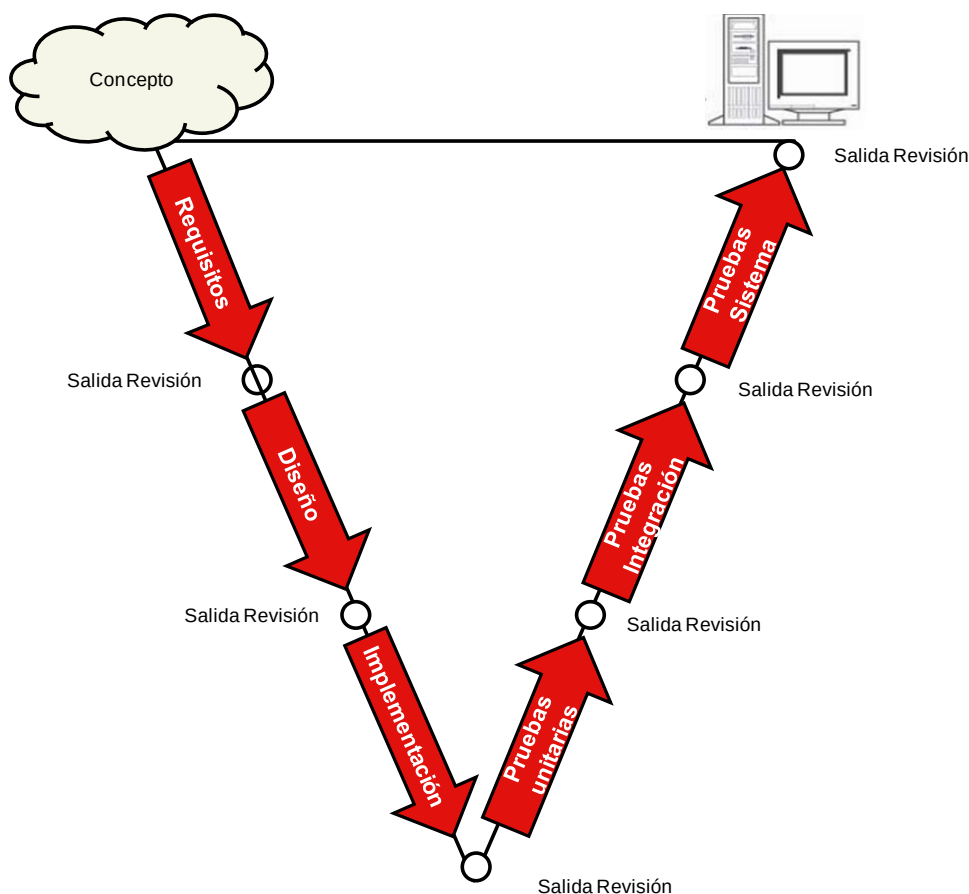
3.2.6. Revisiones

Las revisiones representan la primera forma de pruebas que puede llevarse a cabo durante el ciclo de vida de desarrollo de un producto, y como ya se comentó ayudan a identificar defectos antes de que lleguen a formar parte del código ejecutable.

Revisar código según **estándares de desarrollo** puede también prevenir la aparición de defectos durante la ejecución de pruebas aunque, en este caso, como el código ya está escrito, no se podrán evitar todos los costes y retrasos adicionales.

La revisión es una tarea relacionada fundamentalmente con el proceso de verificación. Se pueden realizar revisiones sobre los requisitos, comprobando si son claros, si hay contradicciones,..., o revisiones durante la fase del diseño sobre el documento de diseño, comprobando si está orientado a los requisitos, por ejemplo. Durante la fase de desarrollo también se pueden realizar revisiones del propio código. Es decir, la revisión es una actividad que puede llevarse a cabo durante todo el ciclo de vida de desarrollo de un producto, incluso es una técnica que puede utilizarse para comprobar si se están llevando a cabo todas las tareas de cada fase del ciclo de desarrollo, si lo están haciendo de la manera correcta, en el orden correcto,...

Figura 14. Revisiones durante el desarrollo de un producto



Todas las revisiones, tanto formales como informales, siguen el mismo proceso básico:

- Identificar los entregables a revisar.
- Desarrollar la lista de participantes en la revisión.
- El documento bajo revisión es estudiado por los revisores.
- Los revisores identifican los problemas o temas a tratar en la revisión y se lo comunican al autor de forma verbal o mediante un documento.
- El autor responde a los comentarios y actualiza el documento según corresponda.

Este proceso básico está siempre presente, pero la mayoría de revisiones formales incluyen etapas adicionales, y prestan más atención a la documentación y a la medición.

Muchas organizaciones sobreestiman los costes que suponen las revisiones y subestiman los beneficios que se pueden obtener de las mismas, y por esa razón no las llevan a cabo. Es cierto que incluso en las revisiones más sencillas, hay que invertir un tiempo en realizarlas, lo que supone un coste. Y en enfoques más formales, el coste de pruebas estáticas también incluye el esfuerzo requerido para recoger y analizar las métricas de la revisión y el coste de implementar las oportunidades de mejora de procesos descubiertas a través de las métricas.

El propósito y beneficios de las revisiones son:

- Mejorar la calidad y el entendimiento de los entregables.
- Identificar y gestionar las expectativas de los involucrados en el negocio.
- Validar que los entregables soporten la solución final.
- Identificar tareas de alto riesgo.
- Formar al equipo de proyecto y a los involucrados en el negocio.
- Detectar problemas relacionados con la calidad de forma temprana en el desarrollo de los entregables.

El número de errores traspasados al proceso de pruebas es uno de los aspectos determinantes respecto al tiempo requerido para completar adecuadamente el proceso de pruebas. Las revisiones tienden a acortar los periodos de pruebas y a reducir el coste de dichas pruebas. A menos tiempo invertido en la corrección de código, mayor productividad habrá en su desarrollo. De hecho, usar revisiones y pruebas a la vez provoca una mejora en la calidad del sistema entregado a los clientes y a los usuarios.

A continuación se va a describir el tipo de revisión denominado *peer review* o revisión entre pares, ya que el modelo CMMI® contempla dichas revisiones entre las prácticas y metas del proceso de verificación, tal y como se mencionó en el apartado referente al modelo.

En el apartado de técnicas de control estáticas se proporciona más información acerca del proceso de revisiones.

3.2.6.1. Peer Reviews

Las *peer reviews* o revisiones entre pares son una parte importante del proceso de verificación. Las revisiones entre pares implican un examen metódico de los productos de trabajo para identificar y eliminar defectos de manera temprana y para recomendar otros cambios que sean necesarios. Se realizan de manera incremental, a medida que los productos de trabajo van siendo desarrollados, por lo tanto son revisiones estructuradas.

Estas revisiones son llevadas a cabo entre pares. Los pares son generalmente compañeros de trabajo en un proyecto que tienen interés y conocimientos acerca del elemento bajo revisión. En general, entre los participantes de la revisión, no deberían estar los gerentes, ya que podrían limitar un diálogo abierto entre el resto de participantes.

Las revisiones entre pares son un método de verificación importante y eficaz implementado vía inspecciones, reuniones de revisión estructuradas u otros métodos de revisión entre compañeros.

Las revisiones entre pares pueden realizarse sobre los productos de trabajo clave de las actividades de especificación, diseño, prueba y de implementación, y sobre los productos de trabajo específicos de la planificación como:

- Artefactos de gestión del proyecto, como los planes que se van generando.
- Artefactos de gestión del proceso, como por ejemplo descripciones de procesos.
- Artefactos de soporte, como definiciones de medidas, documentación y productos de trabajo de formación (normalmente desarrollados por grupos de soporte).

Algunas de las guías a seguir para llevar a cabo una revisión entre pares exitosa son:

- Proporcionar un entorno no amenazante para que pueda desarrollarse una discusión abierta.
- Dar formación al personal sobre los roles de cada uno dentro de la revisión. Estos roles pueden variar de una revisión entre pares a la siguiente.
- Registrar los defectos y problemas en tablas con datos sobre localización del defecto, descripción del defecto (tipo y origen), comentarios y elementos de acción.
- Deben registrarse datos consistentes y suficientes (realizando una inspección formal por ejemplo).

- Gestionar y controlar la revisión por pares.
- El enfoque de la revisión entre pares debería ser sobre el producto de trabajo en revisión, y no sobre la persona que lo produjo.
- Cuando surgen problemas durante la revisión entre pares, deberían comunicarse al desarrollador principal del producto para su corrección.
- Es importante incorporar la planificación de la revisión dentro de los planes y la planificación de desarrollo del proyecto, para ayudar a la asignación adecuada de tiempo y evitar que se salte la preparación de la misma.

4. TÉCNICAS Y HERRAMIENTAS DE VALIDACIÓN Y VERIFICACIÓN

Como ya se ha mencionado con anterioridad, esta guía trata de ser una ayuda para la implementación de las metas de las áreas de procesos Validación y Verificación del modelo CMMI®. Tanto las técnicas como las herramientas que a continuación se describen pueden servir de ayuda y guía para el éxito de la implementación de dichas áreas de proceso.

4.1. TÉCNICAS DE PRUEBAS

Existen distintas técnicas de pruebas de software, con sus debilidades y sus puntos fuertes. Cada una de ellas es buena para encontrar un tipo específico de defectos. Pruebas realizadas en distintas etapas del ciclo de desarrollo del software encontrarán diferentes tipos de defectos. Por ejemplo, es más probable que las pruebas unitarias encuentren defectos lógicos en el código y no en el diseño del sistema.

Las pruebas dinámicas sólo se aplican sobre el código del producto para detectar defectos y determinar atributos de calidad del software, por lo que estarían más orientadas al área de validación. Por otra parte, las técnicas de pruebas estáticas son útiles para evaluar o analizar documentos de requisitos, documentación de diseño, planes de prueba, o manuales de usuario, e incluso para examinar el código fuente antes de ejecutarlo. En este sentido, ayudan más a la implementación del proceso de verificación.

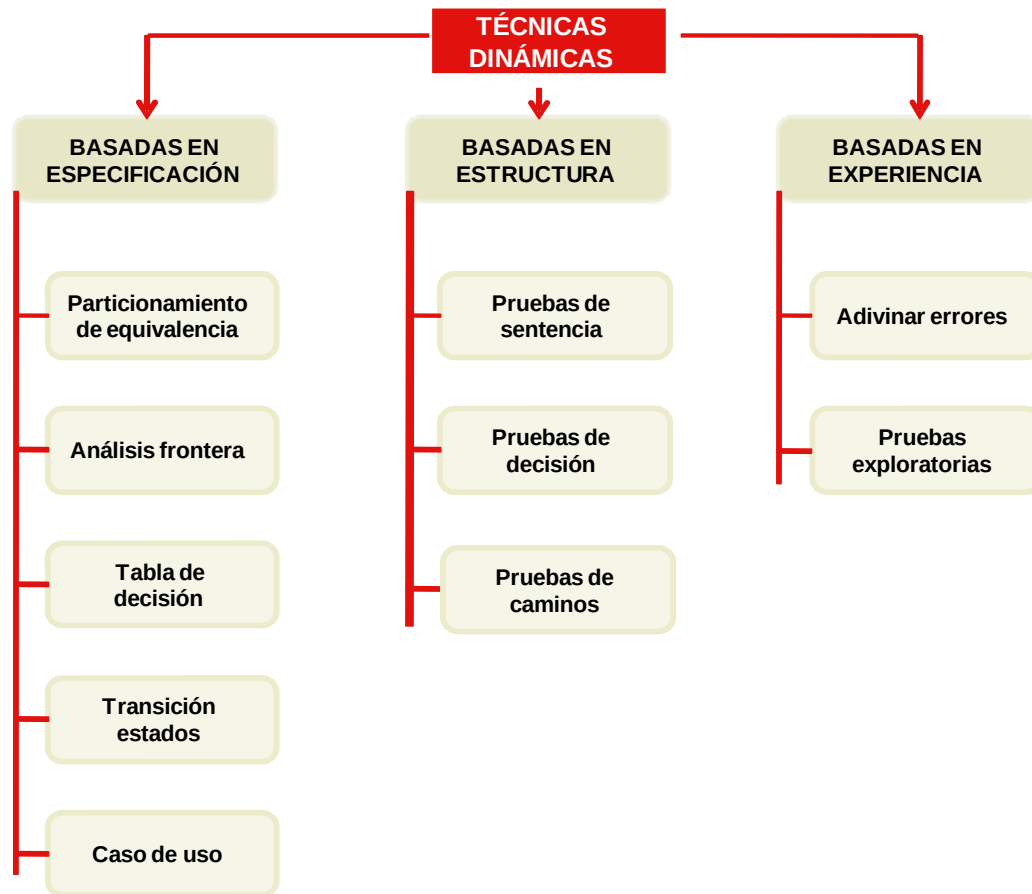
Las pruebas estáticas y las pruebas dinámicas son métodos complementarios ya que ambas tratan de detectar distintos tipos de defectos de forma efectiva y eficiente, aunque las pruebas estáticas están orientadas a encontrar defectos mientras que las dinámicas fallos.

4.1.1. Técnicas de pruebas dinámicas

Las **técnicas de pruebas dinámicas** ejecutan el software. Para ello introducen una serie de valores de entrada, y examinan la salida comparándola con los resultados esperados.

Las técnicas dinámicas se clasifican en: basadas en especificación, también llamadas de caja negra, las técnicas basadas en estructura o de caja blanca y, por último, las técnicas basadas en la experiencia. A continuación, se describe una clasificación de las técnicas tal y como muestra la siguiente figura:

Figura 15. Clasificación técnicas dinámicas



4.1.1.1. Basadas en la especificación

Las técnicas de pruebas dinámicas basadas en la especificación también son conocidas como técnicas de pruebas de **caja negra** o conducidas por entradas/salidas, porque tratan el software como una caja negra con entradas y salidas, pero no tienen conocimiento de cómo está estructurado el programa o componente dentro de la caja. Esencialmente, el técnico de pruebas se concentra en qué hace el software y no en cómo lo hace. Las técnicas basadas en especificaciones obtienen los casos de prueba directamente de las especificaciones o de otros tipos de modelos que contengan lo que el sistema debería hacer.

A continuación, se detallarán las técnicas basadas en la especificación más comunes.

4.1.1.1.1. Particionamiento de equivalencia

Esta técnica consiste en dividir un conjunto de condiciones de prueba en grupos o conjuntos que puedan ser considerados iguales por el sistema. Esta técnica requiere probar sólo una condición para cada partición. Esto es así porque se asume que todas las condiciones de

una partición serán tratadas de la misma manera por el software. Si una condición en una partición funciona, se asume que todas las condiciones en esa partición funcionarán. De la misma forma, si una de las condiciones en una partición no funciona, entonces se asume que ninguna de las condiciones en esta partición en concreto funcionará. Si existe tiempo suficiente, se debería probar más de un valor para una partición, especialmente si se quiere confirmar una selección típica de entradas de usuario.

4.1.1.1.2. Análisis de valor frontera

Los errores generados por los programadores tienden a agruparse alrededor de las fronteras. Por ejemplo, si un programa debería aceptar una secuencia de números entre 1 y 10, el error más probable será que los valores justo fuera del rango sean aceptados de forma incorrecta o que los valores justo en los límites del rango sean rechazados. El análisis del valor frontera está basado en probar los valores frontera de las particiones. Al hacer “comprobaciones de rango”, probablemente se esté usando de forma inconsciente el análisis del valor frontera. En esta técnica también se cuenta con fronteras válidas (en las particiones válidas) y fronteras no válidas (en las particiones no válidas).

4.1.1.1.3. Tablas de decisión

Las técnicas de particionamiento de equivalencia y análisis del valor frontera son aplicadas con frecuencia a situaciones o entradas específicas. Sin embargo, si diferentes combinaciones de entradas dan como resultado diferentes acciones, resulta más difícil usar las técnicas anteriores.

Las especificaciones suelen contener reglas de negocio para definir las funciones del sistema y las condiciones bajo las que cada función opera. Las decisiones individuales son normalmente simples, pero el efecto global de las condiciones lógicas puede ser muy complejo. Los técnicos de pruebas necesitan ser capaces de asegurarse que todas las combinaciones de las condiciones que puedan ocurrir han de ser probadas, por lo tanto, hay que capturar todas las decisiones de una forma que permita explorar sus combinaciones. El mecanismo usado con frecuencia para capturar las decisiones lógicas es la tabla de decisión.

Una tabla de decisión lista todas las condiciones de entrada que pueden ocurrir y todas las acciones que pueden surgir de ellas. Están estructuradas por filas, con las condiciones en la parte de arriba de la tabla y las posibles acciones en la parte de abajo. Las reglas de negocio, que incluyen combinaciones de condiciones para producir algunas combinaciones de acciones, se incluyen en la parte de arriba de un extremo a otro. Cada columna, por lo tanto, representa una regla de negocio individual y muestra cómo se combinan las condiciones de entrada para producir acciones. De esta manera, cada columna representa un posible caso de prueba, ya que identifica ambas entradas y salidas.

4.1.1.1.4. Transición de estados

La técnica anterior (tabla de decisión) es particularmente útil cuando las combinaciones de condiciones de entrada producen varias acciones. La técnica de transición de estados se

utiliza con sistemas en los que las salidas son desencadenadas por cambios en las condiciones de entrada, o cambios de “estado”.

Las pruebas de transición de estados se usan cuando alguno de los aspectos del sistema se pueden describir en lo que se denomina una “máquina de estados finitos”. Esto significa que el sistema puede estar en un número (finito) de estados, y las transiciones de un estado a otro están determinadas por las reglas de la “máquina”. Este es el modelo en el que se basan el sistema y las pruebas. Cualquier sistema en el que se puedan conseguir diferentes salidas con la misma entrada, dependiendo de lo que haya pasado antes, es un sistema de estados finitos. Un sistema de estados finitos se suele representar mediante un diagrama de estados.

4.1.1.1.5. *Pruebas de Casos de uso*

Los casos de uso son una forma de especificar la funcionalidad como escenarios de negocio o flujos de procesos.

Las pruebas de caso de uso son una técnica que ayuda a identificar casos de prueba que ejerciten el sistema entero transición a transición desde el principio al final.

Un caso de uso es una descripción de un uso particular del sistema por un actor (un usuario del sistema). Cada caso de uso describe las interacciones que el actor tiene con el sistema para conseguir una tarea concreta (o, al menos, producir algo de valor al usuario). Los actores son generalmente gente pero pueden ser también otros sistemas. Resumidamente, los casos de uso son una secuencia de pasos que describen las interacciones entre el actor y el sistema.

Los casos de uso están definidos en términos del actor, no del sistema, describiendo qué hace y ve el actor, más que qué entradas o salidas espera el sistema. De forma muy frecuente usan el lenguaje y términos de negocio más que términos técnicos, especialmente cuando el actor es un usuario de negocio. Sirven como fundamento para desarrollar casos de prueba, en su mayoría, en los niveles de pruebas de sistema y pruebas de aceptación.

Los casos de uso pueden descubrir defectos de integración, defectos causados por la interacción incorrecta entre diferentes componentes.

Los casos de uso describen el flujo de proceso a través de un sistema basado en su uso más probable. Esto hace que los casos de prueba obtenidos de los casos de uso sean particularmente útiles a la hora de encontrar defectos en el uso real del sistema (p.ej. los defectos que los usuarios son más propensos a hacer la primera vez que usan el sistema). Cada caso de uso tiene un escenario dominante (o más probable) y algunas ramas alternativas (cubriendo, por ejemplo, casos especiales o condiciones excepcionales). Cada caso de uso debe especificar cualquier precondition que tenga resultados observables y una descripción del estado final del sistema después de que el caso de uso haya sido ejecutado de forma exitosa.

4.1.1.2. Basadas en la estructura

Las pruebas estructurales son una aproximación al diseño de casos de prueba donde las pruebas se derivan a partir del conocimiento de la estructura e implementación del software. Esta aproximación se denomina a veces pruebas de **caja blanca**.

La comprensión del algoritmo utilizado en un componente puede ayudar a identificar particiones adicionales y casos de prueba, asegurando así un mayor rango de pruebas. Las técnicas más sofisticadas proporcionan, de forma incremental, una cobertura de código completa. A continuación, se detallarán las técnicas basadas en la estructura más comunes.

4.1.1.2.1. Pruebas de sentencia

El objetivo de las pruebas de sentencia es ir probando las distintas sentencias a lo largo del código. Si se prueban todas y cada una de las sentencias ejecutables del código, habrá una cobertura de sentencia total. Es importante recordar que estas pruebas sólo se centran en sentencias ejecutables a la hora de medir la cobertura. Es muy útil el uso de gráficos de flujo de datos para identificar este tipo de sentencias, que se representan mediante rectángulos.

Generalmente, la cobertura de las sentencias es demasiado débil para ser considerada una medida adecuada para probar la efectividad.

4.1.1.2.2. Pruebas de decisión

El objetivo de estas pruebas es asegurar que las decisiones en un programa son realizadas adecuadamente. Las decisiones son parte de las estructuras de selección e iteración, por ejemplo, aparecen en construcciones tales como: IF THEN ELSE, o DO WHILE, o REPEAT UNTIL. Para probar una decisión, es necesario ejecutarla cuando la condición que tiene asociada es verdadera y también cuando es falsa. De esta forma, se garantiza que todas las posibles salidas de la decisión se han probado.

Al igual que las pruebas de sentencias, las pruebas de decisión tienen una medida de cobertura asociada e intentan conseguir el 100% de la cobertura de decisión.

4.1.1.2.3. Pruebas de caminos

Las pruebas de caminos son una estrategia de pruebas estructurales cuyo objetivo es probar cada camino de la ejecución independientemente. En todas las sentencias condicionales se comprueban los casos verdadero y falso. En un proceso de desarrollo orientado a objetos, pueden utilizarse las pruebas de caminos cuando se prueban los métodos asociados a los objetos.

Las pruebas de caminos no prueban todas las posibles combinaciones de todos los caminos en el programa. Para cualquier componente distinto de un componente trivial sin bucles, este es un objetivo imposible. Existe un número infinito de posibles combinaciones de caminos en los programas con bucles. Incluso cuando todas las sentencias del programa se han ejecutado al menos una vez, los defectos del programa todavía pueden aparecer cuando se combinan determinados caminos.

4.1.1.3. Basadas en la experiencia

Las técnicas basadas en experiencia son aquellas a las que se recurre cuando no hay una especificación adecuada desde la que crear casos de prueba basados en especificación, ni hay tiempo suficiente para ejecutar la estructura completa del paquete de pruebas.

Las técnicas basadas en experiencia usan la experiencia de los usuarios y de los técnicos de pruebas para determinar las áreas más importantes de un sistema y ejercitar dichas áreas de forma que sean consistentes con el uso que se espera que tengan.

Existe un gran rango de técnicas de pruebas, desde más sofisticadas a más simples, pero en todas ellas el conocimiento y experiencia de los técnicos de pruebas son factores decisivos para el éxito de las mismas.

4.1.2. Técnicas de control estáticas

Aunque las **técnicas estáticas** son usadas cada vez más, las pruebas dinámicas siguen siendo las técnicas predominantes de validación y verificación. Las técnicas estáticas permiten encontrar las causas de los defectos. Aplicar un proceso de pruebas estáticas sobre el proceso de desarrollo permite una mejora del proceso evitando cometer errores similares en el futuro.

Las técnicas de pruebas estáticas proporcionan una forma de mejorar la productividad del desarrollo del software. El objetivo fundamental de las pruebas estáticas es mejorar la calidad de los productos software, ayudando a los ingenieros a reconocer y arreglar sus propios defectos en etapas tempranas del proceso de desarrollo. Aunque las técnicas de este tipo de pruebas son muy efectivas, no conviene que sustituyan a las pruebas dinámicas.

Estas pruebas, son las primeras que se aplican al software y buscan defectos sin ejecutar código. Por esta razón, también se llaman “técnicas de no ejecución”. Las técnicas estáticas tienen que ver con el análisis y control de las representaciones del sistema, es decir de los diferentes modelos construidos durante el proceso de desarrollo de software.

La mayoría de las técnicas estáticas son técnicas de verificación que pueden probar cualquier tipo de documentación ya sea código fuente, o documentación y modelos de diseño, o especificación funcional o de requisitos. La siguiente tabla muestra algunos de los entregables afectados por estas pruebas y los distintos tipos de defectos que se suelen encontrar:

Tabla 5. Entregables afectados por las técnicas estáticas

Entregables afectados	Defectos encontrados
Especificación de requisitos	Desviaciones respecto a estándares
Documentos de diseño, incluyendo diagramas UML, arquitecturas de redes, diagramas de flujo de datos del sistema, y esquemas de la base de datos.	Defectos en los requisitos: por ejemplo, los requisitos son ambiguos. Defectos del diseño: por ejemplo, que el diseño no encaja con los requisitos.
Documentación de usuario o del programador	
Código fuente	Difícil mantenibilidad: por ejemplo, que el código sea muy complejo de mantener.
Modelos del sistema	Especificaciones inconsistentes e incorrectas: por ejemplo, la especificación de las conexiones no se corresponde con el diseño.

Otras ventajas representativas del uso de las técnicas estáticas son:

- El esfuerzo de realizar re-trabajo se reduce sustancialmente, mientras que la productividad de desarrollo probablemente aumente.
- La evaluación realizada por un equipo tiene la ventaja adicional de que hay un intercambio de información entre participantes.
- Las pruebas estáticas contribuyen a un aumento en la conciencia sobre temas de calidad.

Hay distintas técnicas estáticas. Las basadas en examen manual o revisiones o las automatizadas como el análisis estático del código o documentación.

4.1.2.1. Revisiones

Las revisiones son una **técnica estática** que consiste en realizar un análisis de un documento con el objetivo de encontrar y eliminar errores. Este tema ya se introdujo en apartados anteriores, y lo que se pretende en este apartado es profundizar más en el tema.

Las revisiones se usan para comprobar que documentos escritos tales como especificaciones de requisitos, diseño de sistemas, código, planes de pruebas y casos de prueba son correctos. La práctica de revisar documentos (especificaciones) al principio del ciclo de vida del producto ayuda a identificar defectos antes de que lleguen a formar parte del código ejecutable, de tal forma que serán más fáciles y baratos de eliminar. Es decir, identificar errores en las primeras fases del ciclo de vida del producto reduce el coste de corrección de los errores. Si un mismo defecto es encontrado durante la ejecución de **pruebas dinámicas**, esto supondrá el coste extra de crear y probar el código defectuoso,

diagnosticar la causa del defecto, corregir el problema y reescribir el código eliminando el defecto.

A continuación se van a describir distintos tipos de revisiones, y los roles y responsabilidades involucrados en el proceso de revisión. También se identificarán una serie de factores clave para conseguir completar con éxito el proceso de revisión.

4.1.2.1.1. Tipos de revisiones

El tipo de una revisión varía en función de su **nivel de formalidad**. En este caso, '*formalidad*' se refiere a nivel de estructura y documentación asociada con la revisión. Unos tipos de revisión son completamente **informales** mientras que otros son muy **formales**.

No existe una revisión perfecta, sino que cada una tiene un propósito distinto durante las etapas del ciclo de vida del documento. Los principales tipos de revisiones se describen a continuación.

Informal

Dar un borrador de un documento a un colega para que lo lea es el ejemplo más simple de revisión. Es una forma de conseguir beneficios (limitados) a un bajo costo ya que no siguen ningún proceso formal a seguir. La revisión puede implementarse mediante '**pares de programadores**', donde uno revisa el código del otro, o mediante un técnico que lidere el diseño de las revisiones y el código.

Walkthrough

Un walkthrough se caracteriza porque el autor del documento bajo revisión va guiando al resto de participantes a través del documento exponiendo sus ideas para conseguir un entendimiento común y recoger respuestas. Es especialmente útil si los asistentes no están relacionados con el software, o no son capaces de entender los documentos de desarrollo de software. En las revisiones walkthrough se explica el contenido del documento paso a paso hasta llegar a un consenso en los cambios y en el resto de información. La reunión es dirigida por los autores y a menudo está presente un documentador, y para facilitar su ejecución pueden usarse **escenarios** y **simulaciones** para validar el contenido.

Revisión técnica

Una revisión técnica es una reunión que se centra en conseguir consenso sobre el contenido técnico de un documento, por lo que es posible que sea dirigida por un experto técnico. Los expertos necesarios para una revisión técnica son, por ejemplo, responsables del diseño y usuarios clave.

El objetivo principal de este tipo de revisiones es evaluar el valor de conceptos técnicos y alternativas del entorno del proyecto y del propio producto. Este tipo de revisión a menudo se lleva a cabo por "pares" (*peer reviews*) y para facilitar su ejecución suelen utilizarse listas de comprobación o listas de registro.

Inspección

La inspección es el tipo de revisión más formal. El documento bajo inspección es preparado y validado minuciosamente por revisores antes de la reunión, se compara el producto con sus fuentes y otros documentos, y se usan listas de comprobación. En la reunión de la inspección se registran los defectos encontrados y se pospone toda discusión para la **fase de discusión**. Todo esto hace que la inspección sea una reunión muy eficiente.

La inspección es dirigida normalmente por un moderador formado, no por el propio autor del documento, quien realiza un seguimiento formal aplicando criterios de salida. Los defectos encontrados son documentados en una lista de registro, y de manera opcional, para mejorar los procesos y aprender de los defectos encontrados, se realiza un análisis de las causas de los mismos. También es habitual tomar métricas que posteriormente son agrupadas y analizadas para optimizar el proceso.

4.1.2.1.2. Roles y responsabilidades

Los participantes de cualquier tipo de revisión formal deberían tener conocimiento sobre el proceso de revisión. Un factor crítico de éxito de una inspección o revisión técnica es formar a los participantes que van a tomar parte en la misma.

Las mejores revisiones formales surgen de equipos bien organizados, y guiados por un moderador formado. Dentro del equipo de revisión, se distinguen 5 tipos de participantes:

- **Moderador.** El moderador de la revisión dirige el proceso de revisión. En cooperación con el autor del documento, el moderador determina el tipo de revisión que se va a realizar y la composición del equipo de trabajo. El moderador realiza una validación de entrada y un seguimiento del trabajo realizado con el objetivo de controlar la calidad del proceso de revisión.
- **Autor.** Es la persona que ha creado el documento bajo revisión. El objetivo principal del autor es mejorar la calidad del documento y su propia habilidad para escribir futuros documentos.
- **Documentador.** Es la persona encargada de tomar nota de cada defecto mencionado y de cualquier sugerencia acerca de la mejora del proceso aunque, en la práctica, es el autor quien juega este papel.
- **Revisor.** La tarea del revisor, también llamado validador o inspector, es la de validar cualquier material con posibles defectos antes de realizar la propia reunión de revisión. El nivel de minuciosidad, el conocimiento del revisor sobre el dominio o su propia habilidad técnica dependen del tipo de revisión.
- **Supervisor.** Es quien decide destinar un tiempo de la planificación del proyecto para realizar las revisiones, quien determina si se han cumplido los objetivos de la revisión y, por supuesto, quien decide ejecutarla. El supervisor también se ocupa de las solicitudes de formación de los participantes en la revisión.

4.1.2.1.3. Factores de éxito de una revisión

Implementar una revisión (formal) no es algo fácil, ya que hay muchos caminos que llevan al fracaso. La siguiente lista contiene un número de factores de éxito que mejoran las oportunidades de tener éxito a la hora de implementar las revisiones, y que por tanto deberían considerarse:

- Cada revisión debería tener un objetivo claramente predefinido y previamente acordado, y una persona que asegurase que el objetivo se cumple. Esta persona necesitará tener una serie de cualidades, tales como experiencia o pensamiento crítico, para guiar a los moderadores y al resto de los participantes.
- Seleccionar los documentos que sean más críticos, como los relacionados con los requisitos o la arquitectura del producto, beneficiará el proceso de revisión del proyecto.
- Es importante estar seguro de que cada revisión tiene un objetivo claro, y que se ha seleccionado el tipo correcto de revisión que encaje con el objetivo definido, no revisar todo mediante inspecciones.
- Asegurarse que las horas invertidas en el proceso de revisión estén plasmadas en el plan del proyecto y que los ingenieros involucrados en las revisiones han de sido informados de esta planificación con suficiente antelación para prepararse.
- Hacer un seguimiento del tiempo invertido durante las revisiones ayudará a planificar y a gestionar las revisiones de una forma eficiente.
- Es importante proporcionar formación sobre las técnicas de revisión, especialmente sobre las técnicas más formales como las inspecciones. Los aspectos psicológicos también deberían ser un tema importante de formación, de tal forma que la revisión sea una experiencia positiva para el autor.
- Hacer que el proceso sea tan formal como la propia cultura del proyecto y el nivel de madurez permitido, siempre siguiendo las reglas formales.
- La mejora continua del proceso y las herramientas basadas en las ideas de los participantes aseguran la motivación de los ingenieros involucrados. La motivación es la clave para tener un proceso de cambios exitoso.
- Informar de los resultados y beneficios obtenidos del proceso de revisión tan pronto como sea posible, y discutir las consecuencias de los defectos si no han sido encontrados en etapas tempranas.

4.1.2.2. Análisis estático

El análisis estático, al igual que las revisiones, busca defectos sin ejecutar el código. Sin embargo, el análisis estático se lleva a cabo una vez que se escribe el código. Su objetivo es encontrar defectos en el código fuente y en los modelos del software.

Para llevar a cabo el análisis estático se pueden utilizar analizadores estáticos, que son herramientas de software para procesar código fuente. Éstas analizan sintácticamente el programa y tratan de descubrir condiciones potencialmente erróneas y llamar la atención del equipo de validación y verificación. Son una ayuda muy efectiva en las inspecciones.

Las inspecciones de software son muy utilizadas en el análisis estático. Generalmente las inspecciones de software se centran en código fuente pero puede inspeccionarse cualquier representación legible del software como los requerimientos o un modelo de diseño. Cuando se inspecciona un sistema se utiliza conocimiento del sistema, su dominio de aplicación y el lenguaje de programación o modelo de diseño para descubrir errores.

Las inspecciones de código son un pilar fundamental para la construcción de un software de calidad. Las inspecciones ayudan no sólo a conseguir un código eficiente y mejorar así la calidad final del producto, sino que también ayudan a ahorrar tiempo mejorando la productividad. Entre las ventajas fundamentales de las inspecciones frente a las pruebas se destacan las siguientes:

- Durante las pruebas, los errores pueden enmascarar otros errores. Cuando se descubre un error nunca se puede estar seguro de si otras anomalías de salida son debidas a un nuevo error o son efectos laterales del error original. Debido a que la inspección es un proceso estático, no hay que preocuparse de las interacciones entre errores.
- Pueden inspeccionarse versiones incompletas de un sistema sin costes adicionales. Si un programa está incompleto, se necesita desarrollar software de soporte especializado para las pruebas a fin de probar aquellas partes que están disponibles, lo que lógicamente añade costes al desarrollo del sistema.
- Además de buscar los defectos en el programa, una inspección también puede considerar atributos de calidad más amplios de un programa tales como grado de cumplimiento con los estándares, portabilidad y mantenibilidad. Pueden buscarse ineficiencias, algoritmos no adecuados y estilos de programación que podrían hacer que el sistema fuese difícil de mantener y actualizar.

Existen distintos tipos de análisis sintáctico, entre los que se encuentran:

- **Análisis de flujo de control.** Comprueba los bucles con múltiples puntos de entrada o salida, encuentra códigos inalcanzables.
- **Análisis de uso de los datos:** Detecta variables no inicializadas, variables escritas dos veces sin que intervenga una asignación, variables que se declaran pero nunca se usan,...
- **Análisis de interfaz.** Comprueba la consistencia de una rutina, las declaraciones del procedimiento y su uso.

- **Análisis de flujo de información.** Identifica las dependencias de las variables de salida. No detecta anomalías en sí pero resalta información para la inspección o revisión del código.
- **Análisis de caminos.** Identifica los caminos del programa y arregla las sentencias ejecutadas en el camino. Es potencialmente útil en el proceso de revisión.

4.2. HERRAMIENTAS DE PRUEBAS

Este apartado va dirigido a conocer los beneficios que pueden aportar herramientas dedicadas a pruebas dentro de la organización. También se verán los riesgos que supone el uso de herramientas en las organizaciones y una clasificación de distintos tipos de herramientas de pruebas en base de su funcionalidad.

4.2.1. Beneficios del uso de herramientas

Realizar el mismo trabajo de manera repetitiva y manualmente puede ser una tarea tediosa. Las personas tienden a aburrirse y por esa razón aumenta la probabilidad de que cometan errores al realizar la misma tarea una y otra vez. Este es el caso de las pruebas de regresión, por ejemplo, donde se introducen los mismos datos de prueba una y otra vez, comparando los resultados contra los estándares de codificación o creando bases de datos de pruebas específicos.

Mediante la utilización de herramientas se puede conseguir reproducir cierto trabajo previamente realizado, obteniendo resultados consistentes. Esta característica es especialmente útil para confirmar que un defecto se ha arreglado, o para introducir datos de entrada a pruebas, por ejemplo.

En ocasiones, si una persona calcula un valor a partir de informes de incidencias, por ejemplo, puede que omita algo sin darse cuenta o que interprete información de forma incorrecta (subjetividad). El uso de una herramienta consigue que esa predisposición a interpretar de forma subjetiva los datos se elimine y de esta forma la evaluación sea realizada de forma más consistente.

Por otro lado, el tener muchos datos no implica que la información sea comunicada de la manera correcta ni entendida por las personas interesadas. La información que se presenta de forma visual es mucho más sencilla de retener e interpretar para la mente humana por lo que el uso de gráficos es una forma mucho mejor de comunicar información que una larga lista de números. Existen herramientas de propósito especial que ofrecen estas funcionalidades de forma directa a partir de la información que procesan. Algunos ejemplos del uso de gráficos y estadísticas a la hora de mostrar la información se pueden encontrar en herramientas que permiten mostrar el progreso de las pruebas, las tasas de incidencias y rendimiento.

En definitiva, los principales beneficios que aporta el uso de herramientas como medio para llevar a cabo el proceso de pruebas son:

- Reducir el trabajo repetitivo
- Mejorar la consistencia
- Facilitar las evaluaciones objetivas
- Facilitar el acceso a información relacionada con pruebas

Además de los beneficios generales que se han descrito, cada tipo de herramienta ofrece unos beneficios específicos relacionados con el aspecto de las pruebas sobre el que la herramienta trabaja de forma particular.

4.2.2. Riesgos del uso de herramientas

Introducir algo nuevo en una organización, en este caso una herramienta, no suele ser tarea sencilla, ya que siempre habrá problemas técnicos a los que sobreponerse. El uso por primera vez de una herramienta no siempre se hará de la mejor forma y consiguiendo los mejores resultados posibles. Es necesario invertir tiempo para desarrollar métodos a seguir por la organización y así conseguir el máximo provecho posible de la herramienta.

Otro factor importante que hay que considerar a la hora de hacer uso de herramientas durante el proceso de pruebas es la habilidad y conocimiento necesario para realizar buenas pruebas y para usar las herramientas de forma correcta.

Aunque pueden obtenerse grandes beneficios del uso de herramientas durante el proceso de pruebas, es necesario tener en cuenta los riesgos que puede acarrear el uso de herramientas, independientemente de su uso específico. Uno de los más relevantes es tener expectativas poco realistas de la herramienta. Por ello, es importante tener claro qué puede hacer la herramienta, y en función de esto establecer objetivos realistas. Otros riesgos relacionados son:

- Subestimación del tiempo, coste y esfuerzo necesario para la introducción inicial de la herramienta en la organización
- Subestimación del tiempo y esfuerzo necesario para conseguir beneficios significantes y continuos de la herramienta
- Subestimación del esfuerzo requerido para mantener los activos de pruebas generados por la herramienta
- Sobre-confianza en la herramienta

Para introducir una herramienta en una organización habría que comenzar trabajando con la organización, más que con la herramienta. Las organizaciones necesitan estar preparadas para los cambios que supondrá la nueva herramienta. Si, por ejemplo, las prácticas que sigue una organización al ejecutar las pruebas no son buenas y la organización no es

madura, por lo general sería más eficiente mejorar dichas prácticas que buscar herramientas que den soporte a este tipo de prácticas.

En ocasiones, se pueden mejorar los propios procesos de la organización de manera paralela a la inserción de una herramienta que de soporte a dichos procesos. Es importante tener claro que la herramienta no debería dirigir sino proporcionar soporte a lo que la organización haya definido.

A la hora de introducir una herramienta en una organización es importante:

- Evaluar la madurez de la organización
- Identificar las áreas de la organización donde las herramientas puedan ayudar y dar soporte a los procesos de prueba
- Evaluar herramientas contra requisitos claros y criterios objetivos
- Planificar la implementación interna de la herramienta
- Llevar a cabo una prueba de concepto para comprobar si el producto funciona como se desea y cumple los requisitos y objetivos definidos. Para ello se puede utilizar un **proyecto piloto**.

Los objetivos de un proyecto piloto para una nueva herramienta son aprender más sobre la herramienta, decidir maneras estándar de usar la herramienta para usuarios potenciales, ver cómo la herramienta se adapta a los procesos o documentación existentes y cómo estos procesos deberían cambiar para trabajar bien con la herramienta y, en definitiva, evaluar el proyecto piloto frente a los objetivos establecidos.

4.2.3. Clasificación de herramientas de pruebas

A continuación se definen una serie de herramientas en base a su funcionalidad. Las herramientas están clasificadas dependiendo de las áreas o actividades de pruebas a las que se enfocan. La mayoría de las herramientas comerciales pueden usarse para diferentes funciones. Por ejemplo, una herramienta de gestión puede proporcionar soporte para la gestión de pruebas, gestión de la configuración, gestión de incidencias y gestión de los requisitos y trazabilidad.

4.2.3.1. Herramientas de soporte para la gestión de pruebas

La gestión de pruebas se aplica sobre el conjunto del ciclo de vida de desarrollo del software, por lo que una herramienta de gestión de pruebas debería de ser la primera en usarse en el proyecto. Esta área abarcaría cuatro tipos de pruebas:

Herramientas de gestión de pruebas

Las herramientas de gestión de pruebas ayudan a recoger, organizar y comunicar información sobre las pruebas en un proyecto. Esta información puede usarse para monitorizar el proceso de pruebas y decidir las acciones a tomar. Asimismo, estas herramientas también proporcionan información sobre el componente o sistema que es probado.

Herramientas de gestión de requisitos

Las herramientas de gestión de requisitos, a pesar de no ser herramientas de gestión de pruebas como tal, son útiles para las pruebas dado que éstas se basan en los requisitos. De esta forma, cuanto mayor sea la calidad de los requisitos, más fácil será desarrollar pruebas para comprobar que son correctos. Asimismo, es importante mantener una trazabilidad bidireccional entre los requisitos y las pruebas, y estas herramientas pueden ser de gran ayuda en esta tarea.

Herramientas de gestión de incidencias

Las herramientas de gestión de incidencias facilitan el mantenimiento del registro de incidencias a lo largo del tiempo. Este tipo de herramientas permiten identificar defectos, problemas, anomalías o posibles mejoras.

Herramientas de gestión de configuración

Las herramientas de gestión de configuración no son estrictamente herramientas de pruebas, sin embargo, una buena gestión de configuración es crítica para controlar las pruebas. Es necesario conocer exactamente qué debe ser probado así como la versión exacta de todos los componentes del sistema.

4.2.3.2. Herramientas de soporte para pruebas estáticas

Las herramientas de las que se habla en este apartado pueden dar soporte a las actividades de prueba descritas en el apartado relativo a las pruebas estáticas. Esta área abarcaría tres tipos de pruebas:

Herramientas de revisión de procesos

Las herramientas de revisión de procesos se hacen especialmente útiles para revisiones formales, donde hay mucha gente involucrada o donde las personas implicadas están en lugares geográficamente separados.

Es cierto que mediante el uso de hojas de cálculo o documentos de texto se puede llevar un seguimiento de toda la información del proceso de revisión, pero también hay que recordar que una herramienta de revisión está diseñada exclusivamente para este propósito, y por lo tanto será más probable conseguir un buen resultado utilizándola. Otra ventaja que ofrece

este tipo de herramientas es que facilitan mucho el trabajo de recogida de información del proceso de pruebas.

Herramientas de análisis estático

Las herramientas de análisis estático normalmente son utilizadas por desarrolladores como parte del proceso de pruebas. La característica principal de estas herramientas es que el código no se ejecuta, sino que sirve de elemento o dato de entrada a la herramienta. Algunas de las características de estas herramientas son que calculan métricas (p.ej., complejidad ciclomática), imponen estándares de codificación, analizan estructuras y dependencias, identifican anomalías en el código, entre otras.

Herramientas de modelado

Las herramientas de modelado ayudan a validar modelos del sistema, por ejemplo, validando la consistencia de los objetos en una base de datos o encontrando inconsistencias y defectos. Estas herramientas son de gran utilidad en el diseño de software.

Una ventaja que tienen tanto las herramientas de análisis estático como las de modelado es que pueden utilizarse antes de la ejecución de las pruebas dinámicas. Esto permite detectar defectos tan pronto como sea posible, lo que supone que arreglarlos sea más sencillo y barato.

4.2.3.3. Herramientas de soporte para la especificación de pruebas

Estas herramientas dan soporte a actividades de pruebas tales como el análisis de pruebas, el diseño de pruebas o la implementación de pruebas. En este apartado destacamos dos tipos de herramientas de soporte para la especificación de pruebas:

Herramientas de diseño de pruebas

Las herramientas de diseño de pruebas son de gran utilidad a la hora de comenzar con el diseño de pruebas, aunque no harán todo el trabajo. El beneficio que ofrece este tipo de herramientas es que pueden identificar de manera rápida y sencilla las pruebas a ejecutar sobre todos los elementos del sistema. Es decir, ayuda a que el proceso de pruebas sea más exhaustivo.

Las herramientas de diseño de pruebas también pueden ayudar a identificar valores de entradas a las pruebas (requisitos, modelos de diseño, condiciones de prueba...), a construir casos de pruebas, a seleccionar los factores a tener en cuenta para asegurar que todos los pares de combinación son probados, etc.

Herramientas de preparación de datos de pruebas

Establecer datos de pruebas puede ser una tarea tediosa, especialmente si se contempla un gran volumen de datos para las pruebas. Las herramientas de preparación de datos de prueba sirven de ayuda en esta área. Estas herramientas suelen usarse por los

desarrolladores, aunque también se utilizan durante las pruebas de sistema y de aceptación. Son especialmente útiles en las pruebas de rendimiento, donde es imprescindible trabajar con gran cantidad de datos realistas.

4.2.3.4. Herramientas para ejecución y registro de pruebas

En este apartado se van a definir tres tipos de herramientas relacionadas con la ejecución y registro de pruebas.

Herramientas de ejecución de pruebas

Cuando las personas piensan en herramientas de pruebas normalmente tienen en mente este tipo de pruebas. La mayoría de las herramientas de este tipo ofrecen un mecanismo para la captura y registro de pruebas.

Estas herramientas suelen utilizar un lenguaje de scripting, por lo que si los técnicos de pruebas desean utilizar una herramienta de ejecución de pruebas necesitarán tener habilidades de programación sobre la creación y modificación de scripts.

Comparadores de pruebas

La esencia de las pruebas es comprobar si el software produce el resultado correcto, y para ello deben comparar lo que el software produce y lo que debería producir. Los comparadores de pruebas ayudan a automatizar estos aspectos. Hay dos maneras de llevar a cabo esta comparación. De manera dinámica, donde la comparación es realizada dinámicamente, es decir, mientras la prueba se está ejecutando, o post- ejecución, donde la comparación se realiza después de que la prueba haya acabado y el software bajo prueba no se vuelva a ejecutar.

Herramientas de seguridad

Hay numerosas herramientas que protegen a los sistemas de ataques externos, por ejemplo, los cortafuegos. Las herramientas de pruebas de seguridad se utilizan para probar la seguridad que tienen los sistemas, intentado acceder a un sistema, por ejemplo. También se encargan de identificar virus, simular ataques externos, detectar intrusos, etc.

4.2.3.5. Herramientas de monitorización y rendimiento

Las herramientas descritas en esta sección soportan pruebas que pueden llevarse a cabo durante el propio proceso de pruebas o después de la liberación del sistema. En esta sección se tratarán dos de estas herramientas, aunque también se podría hablar de herramientas relacionadas con las pruebas de rendimiento, de carga y de estrés definidas en apartados anteriores.

Herramientas de análisis dinámico

Las herramientas de análisis dinámico se llaman así porque son “dinámicas”, es decir, requieren que el código se esté ejecutando, y porque no ejecutan pruebas como tal, sino que analizan lo que ocurre cuando el software se está ejecutando.

Herramientas de monitorización

Las herramientas de monitorización se utilizan para realizar un seguimiento del estado del sistema en uso, con el objetivo de conseguir avisos tempranos de los problemas y de mejorar el servicio. Hay herramientas de monitorización para servidores, redes, bases de datos, seguridad, rendimiento, páginas web y uso de internet, y para aplicaciones.

5. ESTÁNDARES DE REFERENCIA

Aunque al comienzo de esta guía se proporciona una visión de la validación y verificación en el modelo CMMI®, existen otros estándares relacionados con los procesos de validación y verificación. En este apartado veremos alguno de ellos.

5.1. IEEE STD. 1012-2004

El estándar IEEE 1012 proporciona una guía detallada para llevar a cabo las tareas de validación y verificación del software. El estándar describe la validación y verificación de software de la siguiente manera:

- Validación: confirmación mediante examen y evidencias objetivas de que los requisitos particulares para un uso específico propuesto son alcanzados.
- Verificación: confirmación mediante examen y evidencias objetivas de que los requisitos específicos han sido alcanzados.

Este estándar persigue los siguientes objetivos:

- Establecer un marco común para los procesos, actividades y tareas asociadas a la validación y verificación como apoyo y soporte al ciclo de vida del software, abarcando los procesos de adquisición, suministro, desarrollo, operación y mantenimiento.
- Definir las tareas de validación y verificación, así como sus entradas y salidas requeridas.
- Identificar un mínimo de tareas de validación y verificación (V&V) a llevar a cabo, en función de lo que denomina “niveles de integridad” del software. El término “nivel de integridad” es usado en el estándar como medio para cuantificar la criticidad del software y, de esta forma, servir de guía para determinar la intensidad y rigor a aplicar en las tareas de V&V, manteniendo los riesgos dentro de límites aceptables. El estándar permite que dicho nivel y, por tanto, el rigor e intensidad de los esfuerzos de V&V, sea determinado de forma separada para las distintas partes del software y define cuatro niveles: alto, serio, moderado y bajo.
- Especificar el contenido y formato del Plan de Verificación y Validación (SVVP).

En relación con el SVVP, el estándar establece que se deben identificar y describir los métodos a usar para:

- Verificar que los requisitos de las especificaciones del software han sido aprobados por el responsable apropiado, que estos requisitos son trasladados a las descripciones de diseño y que dichas descripciones de diseño se implementan en el código.

- Validar que el código, al ejecutarse, cumple con los requisitos recogidos en las especificaciones de requisitos.

En relación con los procesos, el estándar incluye un conjunto de procesos y para cada uno de ellos especifica unas actividades de V&V asociadas. El estándar establece una única actividad de V&V para cada proceso, a excepción del proceso de desarrollo que comprende varias actividades, las cuales se corresponden básicamente con las fases del ciclo de vida del software. Las actividades propuestas en el estándar con su correspondiente proceso son:

- **Proceso de gestión:** en este proceso, según el estándar, aplica a la actividad de “gestión del esfuerzo de V&V”. Esta actividad se ocupa de evaluar todas las salidas correspondientes a la validación y verificación; de la revisión continua del esfuerzo de V&V, así como del plan de V&V; de la ejecución de revisiones y auditorías; la identificación de oportunidades de mejora en relación con la V&V; etc. La gestión del esfuerzo de V&V se lleva a cabo para todos los procesos y actividades del ciclo de vida del software.
- **Proceso de adquisición:** este proceso incluye la actividad de V&V de “soporte a la adquisición”. Esta actividad cubre el inicio del proyecto, la petición de propuestas (RFP), la preparación del contrato, la monitorización del proveedor y la aceptación y finalización del mismo.
- **Proceso de suministro:** este proceso contempla la actividad de “planificación de V&V”. Esta actividad abarca la preparación de respuestas, el contrato, la planificación, la ejecución y control, la revisión y evaluación y la entrega y finalización de actividades.
- **Proceso de desarrollo:** dentro de este proceso se llevan a cabo una serie de actividades de V&V:
 - V&V del concepto.
 - V&V de requisitos.
 - V&V de diseño.
 - V&V de la implementación.
 - V&V de las pruebas.
 - V&V de la instalación.
- **Proceso de operación:** dentro de este proceso se realiza la actividad de V&V de “operaciones”, la cual se ocupa de evaluar el impacto de los cambios en el entorno operativo, de valorar el efecto de los cambios propuestos en el sistema y de valorar

la adherencia de los procedimientos de operación, y de analizar los riesgos que puedan afectar al usuario y al sistema.

- **Proceso de mantenimiento:** dentro de este proceso se propone la actividad de V&V de “mantenimiento”, estableciendo que las modificaciones del sistema software deben ser tratadas como procesos de desarrollo y, por tanto, verificadas y validadas según las actividades descritas para los procesos de desarrollo y gestión.

Para cada una de las actividades descritas correspondientes a cada proceso, el estándar define un mínimo de tareas de validación y verificación que son función del “nivel de integridad” asignado para el elemento software en cuestión.

5.2. IEEE STD. 829-2008

El estándar IEEE 829 proporciona una base estándar para la documentación del proceso de pruebas, permitiendo plasmar todos los aspectos clave de las pruebas. El estándar propone los siguientes documentos relacionados con las pruebas:

- **Plan de pruebas.** Documento que describe el alcance, el enfoque a seguir, los recursos a asignar y la planificación de las actividades de pruebas. También identifica los elementos de pruebas, las características o funcionalidades a probar, las tareas a realizar en relación con las pruebas, el responsable de ejecutar cada tarea, así como, los principales riesgos a tener en cuenta, en especial las circunstancias bajo las cuales se parará o se reiniciará el proceso de pruebas.
- **Especificaciones de diseño de pruebas.** Documento que especifica las características y funcionalidades que se quieren probar del producto software (condiciones o requisitos de pruebas) y su priorización, así como los criterios de éxito/fallo de las pruebas.
- **Especificación de casos de pruebas.** Documento que especifica aspectos como las entradas, los resultados previstos, las precondiciones fijadas, las interdependencias entre los casos de pruebas, el conjunto de condiciones de ejecución para un elemento de prueba dado, etc.
- **Especificación de procedimientos de pruebas.** Documento que recoge la secuencia de acciones para la ejecución de una prueba y la configuración exacta del entorno de pruebas.
- **Informe de transferencia de elementos de pruebas.** Documento que identifica los elementos de pruebas y contiene información sobre el estado actual e información de localización de dichos elementos. De esta forma, los técnicos de pruebas pueden tener la garantía de que los elementos que van a probar estén preparados y sepan dónde localizarlos.
- **Registro de pruebas.** Registro cronológico de los detalles más relevantes relacionados con la ejecución de las pruebas.

- **Informe de incidencias de pruebas.** Documento que recoge cualquier evento ocurrido durante el proceso de pruebas que requiera ser investigado y resuelto.
- **Informe de resumen de pruebas.** Documento que resume las actividades y resultados de las pruebas. También contiene la evaluación de los elementos de prueba correspondientes.

El estándar especifica que no todos estos documentos son necesarios. La documentación de pruebas a generar depende del tamaño del proyecto, del grado de formalidad requerido por el proyecto o de la naturaleza de las actividades de pruebas.

Este estándar ofrece una aproximación o enfoque modular de la documentación de pruebas que en un principio puede parecer complejo, pero los resultados evidencian que es flexible y efectivo. En conclusión, este estándar apuesta por el desarrollo de diferentes documentos de pruebas durante las distintas fases del ciclo de vida, fomentando la reutilización de procedimientos y casos de pruebas.

5.3. ISO/IEC 29119

La ISO/IEC 29119 es un nuevo estándar de pruebas de software que actualmente está bajo desarrollo. Su cuyo objetivo es producir un estándar internacional para pruebas de software que cubra el ciclo de vida de pruebas íntegramente, es decir que vaya desde el análisis, al diseño, desarrollo y mantenimiento de cualquier producto software o sistema.

Este estándar comprende 4 partes:

- Parte 1: Conceptos y vocabulario
- Parte 2: Proceso de pruebas
- Parte 3: Documentación de pruebas
- Parte 4: Técnicas de prueba

Este nuevo estándar remplazará una serie de estándares actuales IEEE y BSI relacionados con las pruebas de software tales como:

- IEEE 829 - Documentación de pruebas
- IEEE 1008 - Pruebas unitarias
- BS 7925-1 - Vocabulario de términos sobre pruebas del software
- BS 7925-2 - Estándar de pruebas de componentes del software

6. GLOSARIO

- **Análisis de riesgos:** proceso de evaluar riesgos identificados para estimar su impacto y probabilidad de ocurrencia.
- **Análisis estático:** análisis de artefactos software, p.ej. requisitos o código, llevado a cabo sin la ejecución de dichos artefactos.
- **Aseguramiento de la calidad (QA):** conjunto de actividades diseñadas para asegurar que el proceso de desarrollo y/o mantenimiento es adecuado para que el sistema cumpla sus objetivos.
- **Caso de prueba:** conjunto de valores de entrada, precondiciones de ejecución, resultados esperados y post condiciones de ejecución desarrolladas para un objetivo o condición de prueba particular, tal como probar un determinado camino de un programa.
- **Control de calidad (QC):** conjunto de actividades diseñadas para evaluar el producto de trabajo ya desarrollado.
- **Driver:** componente software o herramienta de prueba que reemplaza un componente que se encarga del control y/o de la llamada a un componente o sistema.
- **Informe de incidencias de pruebas:** documento que recoge cualquier evento ocurrido durante el proceso de pruebas que requiera de investigación y resolución.
- **Informe de resumen de pruebas:** documento que resume las actividades y resultados de las pruebas. También contiene la evaluación de los elementos de prueba correspondientes.
- **Peer reviews** (Revisión entre pares): las revisiones entre pares es un método de verificación importante y eficaz implementado vía inspecciones, revisiones técnicas, *walkthrough* u otros métodos de revisión entre compañeros.
- **Plan de pruebas:** documento que describe el alcance, el enfoque a seguir, los recursos a asignar y la planificación de las actividades de pruebas.
- **Prototipo:** borrador de un producto potencial o de una parte del mismo, una simulación de los requisitos.
- **Pruebas:** actividad en la cual un sistema, o uno de sus componentes, se ejecutan en circunstancias previamente especificadas, se observan los resultados, se registran y se realiza una evaluación de mismos.
- **Pruebas dinámicas:** pruebas que implican la ejecución del software de un componente o sistema.

- **Pruebas estáticas:** pruebas de un componente o sistema a nivel de especificación o implementación sin la ejecución de dicho software, p.ej. revisiones o análisis estático.
- **Registro de pruebas:** registro cronológico de los detalles más relevantes relacionados con la ejecución de las pruebas.
- **Revisión:** evaluación del estado de un producto o proyecto para encontrar discrepancias con los resultados planificados y recomendar mejoras. P.ej.: revisión informal, revisiones técnicas, inspecciones,...
- **Riesgos:** factor que puede resultar en consecuencias negativas en el futuro. Normalmente se expresa en términos de impacto y probabilidad.
- **Sistema de pruebas:** Las pruebas forman parte de lo que se denomina sistema de pruebas. Los cuatro componentes principales de un sistema de pruebas son equipo, recursos, procesos y entorno de pruebas.
- **Stub:** implementación esquemática o con un propósito especial de un componente software, usado para desarrollar o probar un componente que llama o depende de él. Reemplaza al componente llamado.
- **Test-driver:** son elementos de prueba que se utilizan para facilitar probar la integración de los componentes de un sistema en cada momento.
- **Tipo de prueba:** grupo de actividades de prueba dirigidas a probar un componente o sistema centrado en un objetivo de prueba específico.
- **Validación:** proceso de comprobar que un sistema cumple las necesidades y expectativas del cliente.
- **Verificación:** proceso de comprobar que un sistema cumple su especificación.
- **Walkthrough:** presentación paso a paso por el autor de un documento para reunir información y para establecer un entendimiento común de su contenido.

7. ACRÓNIMOS

- **AVAL** (*Acquisition Validation*): Validación de la Adquisición
- **AVER** (*Acquisition Verification*): Verificación de la Adquisición
- **CMMI®** (*Capability Maturity Model Integration*): Modelo de Capacidad y Madurez Integrado.
- **CMMI-ACQ®** (*Capability Maturity Model Integration for Acquisition*): Modelo de Capacidad y Madurez Integrado para Adquisiciones.
- **CMMI-DEV®** (*Capability Maturity Model Integration for Development*): Modelo de Capacidad y Madurez Integrado para Desarrollo.
- **CMMI-SVC®** (*Capability Maturity Model Integration for Service*): Modelo de Capacidad y Madurez Integrado para Servicios.
- **IEEE** (*Institute of Electrical and Electronics Engineers*): Instituto de Ingeniería Eléctrica y Electrónica.
- **ISO** (*International Organization for Standardization*): Organización Internacional para la Estandarización.
- **QA** (*Quality Assurance*): Aseguramiento de calidad
- **QC** (*Quality Control*): Control de calidad
- **RFP** (*Request for proposal*): Solicitud de propuesta
- **SVVP** (*Software Verification and Validation Plan*): Plan de Verificación y Validación.

8. REFERENCIAS

- [1] Beizer, B., “*Software Testing Techniques*”, Segunda Edición, 1990
- [2] Black, R., “*Managing the Testing Process*”, Segunda Edición, 2001
- [3] Boehm, B., “*Software Engineering Economics*”, 1981
- [4] Brian Hambling, Peter Morgan, Angelina Samaroo, Geoff Thompson y Peter Williams, *Software Testing - An ISEB Foundation*, 2007
- [5] CMMI® Product Team, *CMMI® For Acquisition, Version 1.2. Improving processes for acquiring better products and services*, 2007
- [6] Dijkstra, Edsger, “Notes on Structured Programming”, 1972
- [7] IEEE Computer Society, *IEEE Std. 829-1998 Software test documentation*, IEEE, 1998.
- [8] IEEE Standard for Software Verification and Validation, IEEE 1012, 2004
- [9] Mary Beth Chrissis, Mike Konrad y Sandy Shrum, *CMMI® Second Edition. Guidelines for Process Integration and Product Improvement*, 2007
- [10] Rex Black, *Pragmatic Software Testing: Becoming an effective and efficient test professional*, 2007
- [11] Roger S. Pressman, *Software Engineering. A practitioner's Approach*, Quinta Edición, 2001
- [12] Ron Burbach, *Software Engineering Methodology*, 1998
- [13] Sommerville, Ian, *Software Engineering 07 Edition*, 2005
- [14] ISO/IEC 29119 Software Testing Standard: <http://www.softwaretestingstandard.org/>